Département d'électronique – Master 2 ESTel

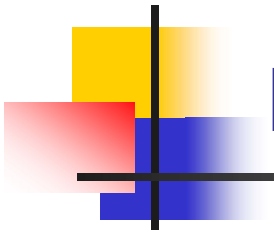
Architectures multi-unités de traitement et multicores

C. Belleudy

(cecile.belleudy@univ-cotedazur.fr)

UNIVERSITÉ CÔTE D'AZUR

DIGITAL SYSTEMS FOR HUMANS
GRADUATE SCHOOL & RESEARCH



PLAN

1. Introduction & paramètres importants
2. Architecture d'un core
    1. Architecture pipeline
    2. Architecture du système mémoire
    3. Architecture Superscalaire et VLIW
3. Architecture Multicores

Evolution des architectures

Augmentation croissante de la capacité de traitements, de mémorisation, de transmission

Unités de traitement (Processeur, ASICs, FPGA) :

- ⇒ Course à la fréquence (pipeline),
- ⇒ Parallélisme : au sein d'un core, multicores, manycores

Le nombre de cores double tous les 2 ans (Patterson, 2007) ?

Mémoire externe :

Capacité x 2 tous les 3 ans

Exemple en téléphonie mobile

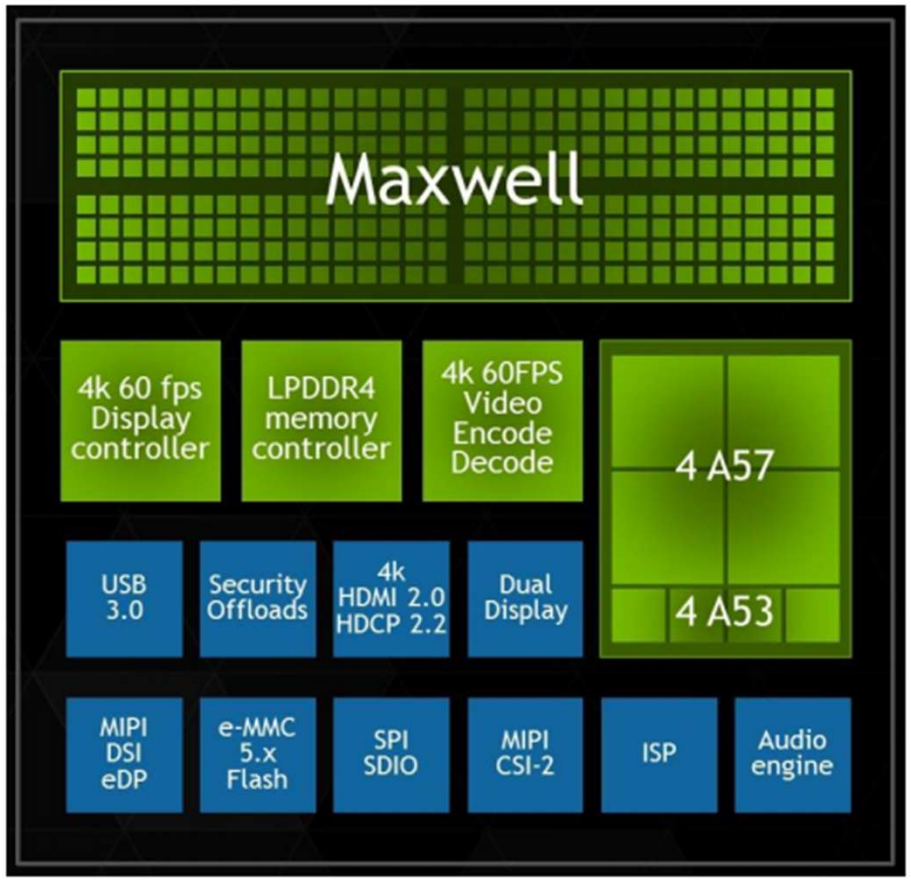
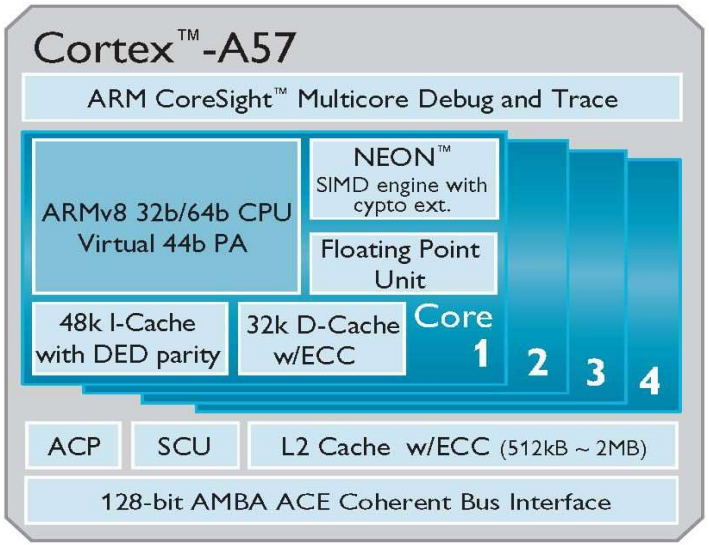
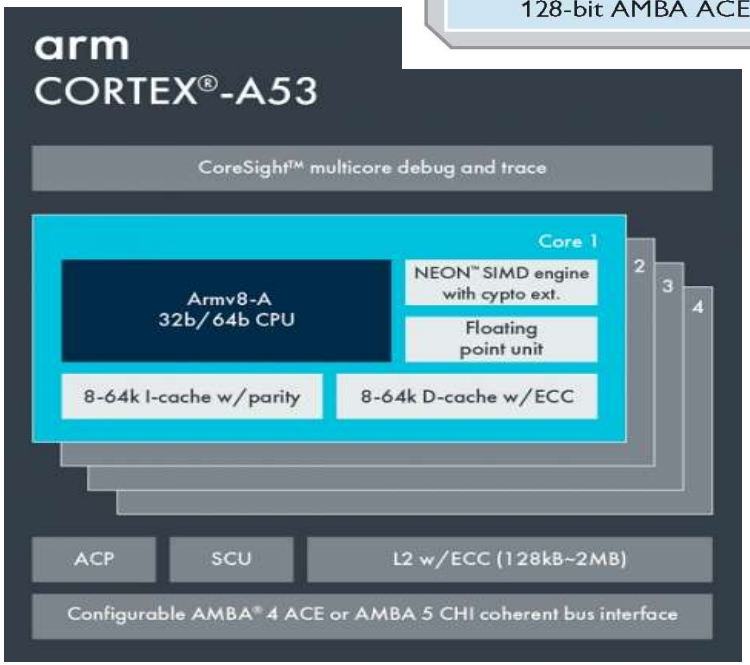


Figure 1 NVIDIA Tegra X1 Mobile Processor





Notion de Performance

Performance

Temps d'exécution, Temps de réponse (running time, execution time)

Notion de Débit (data rate, throughput)

Coder/decoder d'image : 25, 30, 60 frames/s (HD : 10 Mbps, SD : 3 Mbps)

Audio : mp3 – 192-320 Kbps …

Autre - débit réseau :
5Mbps or less: Basic web surfing and email
5-10Mbps: Web surfing, email, occasional streaming and online gaming with few connected devices

10-25Mbps: Moderate HD streaming, online gaming and downloading with a moderate number of connected devices
25-40Mbps: Heavy HD streaming, online gaming and downloading with a lot of connected devices
40+Mbps: Hardcore streaming, gaming, and downloading with an extreme number of connected devices

Performance vs Temps d'exécution

Performance = 1/Temps d'exécution

- Temps d'exécution :
  - s (sec), ms, µs, ns
  - Nombre de cycles machine (doit être multiplié par la fréquence).

- Temps d'exécution pondéré (W_i x $Time_i$)

  Sur plusieurs exécutions : on pondère par une probabilité d'exécution d'un programme (profiling).
  - Exemple : boucle for, if then else, …
  - Application : encoder video, decoder video …

    HD : 1920x1080 => différence entre deux images

Temps d'exécution

Program 1 :

```
FOR (I=1 TO 32)
        Y = Y + X.H;
END FOR;
```

Quel est le temps d'exécution de ces programmes ?

Program 2 :

```
IF A>B Then
        Y = Y + X.H
   ELSE
        N=B-A;
        FOR (I=1 TO N)
        {
        Xn = Xn-1 + Y.H
        }
        END FOR;
END IF;
```

Profiling d'après des "use cases" :

Then : 30%, Else : 70%, N : (10%, 16), (50%,8), (40%,32)

Notion possible de corner case our des applications critiques

Temps d'exécution

- Temps d'exécution normalisé (calculer par rapport à un processeur de référence).

Exemple :

- Program A sur CPU 1 référent : Texe= 1,5 ms, Tnormalisé =1
- Program A sur CPU 2 : Texe = 3 ms, Tnormalisé = 2,
- Program A sur CPU 3 : Texe = 0,75 ms, Tnormalisé = 0,5

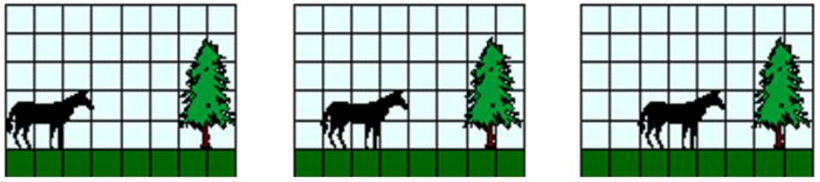
Exercice : compression vidéo

On souhaite calculer l'erreur moyenne (Err) sur un bloc de pixels (8x8) entre deux images consécutives.

Processeur 1 référent : 1 cycle par instruction

Processeur 2 :

Calculer le temps d'exécution normalisé pour un processeur capable de faire de 2 opérations add ou sub par cycle.



Temps d'exécution

- Cas Monoprocesseur ou 1 core : $T_{execution} = T_{CPU} + T_{pénalités}$

  T_{CPU} =Nombre de cycles CPU x 1/F (cas idéal)

  Pénalités : Pipeline stall, Cache miss, communication entre les tâches…

- Autres notations :
  - WCET : worst case execution time,
  - BCET : best case execution time,
  - AET : actual execution time ou average execution time ?

Reprendre l'exercice du slide précédent et rajouter : si Err>seuil alors rechercher le bloc le plus proche dans le macrobloc (16x16). Déterminer WCET, BCET, AET.

- Cas Multicores :
  - Se ramène à un cas monoprocesseur (parallélisme au niveau des thread) sinon $\Sigma_i T_{execution_core_i}$

    Comment peut-on améliorer le calcul de Err avec une architecture multi-core ?

Autre métrique & performance

- Nombre d'instruction par cycle => taux de parallélisme
- CPI (nombre d'instruction par cycle d'horloge, clock rate per instruction) :

$$CPI = \frac{\text{Nombre de cycle pour exécuter un programme}}{\text{Nombre d'instruction à exécuter}}$$

- MIPS (million d'instruction par seconde) :

$$MIPS = \frac{\text{Nombre d'instructio n}}{\left(\text{Temps d'exécution} * 10^{6}\right)}$$

- …
- Teraflops : nombre de tera opérations flottantes par seconde

Notion de performances - comparaison

Comparaison entre deux processeurs : rapport des performances

Attention : Le temps gagné peut ne porter que sur une fraction du temps d'exécution.

EX1 : Considérons un programme P1 qui s'exécute en 300 ms sur un processeur CPU1 à 1 Ghz. Le temps passé sur le CPU est de 60% alors que le temps d'attente pour les I/O est de 40%. Déterminer le speedup si on triple la vitesse de ce processeur.

Ex2 : Reprendre l'exercice 1

Architecture référente : 1CPU, 1 opération/cycle

Déterminer le speedup pour les architectures suivantes :

- Arch 2 : 2 opérations/cycle
- Arch 3 : 2 cores,
- Arch 4 : 4 cores

Précisez les hypothèses

Evolution des architectures

Exigence croissante en termes de traitements, de capacité de mémorisation, de transmission

Unités de traitement (Processeur, ASICs, FPGA) :

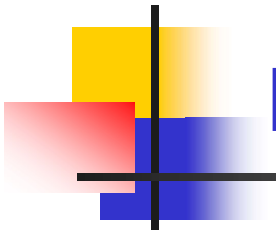
⇒ Course à la fréquence (pipeline),

⇒ Parallélisme : au sein du CPU, multicores, manycores

Le nombre de cores double tous les 2 ans (Patterson, 2007)

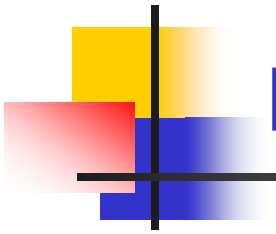
Mémoire externe :

Capacité x 2 tous les 3 ans



PLAN

1. Introduction & paramètres importants
2. Architecture d'un core
    1. Architecture pipeline
    2. Architecture du système mémoire
    3. Architecture Superscalaire et VLIW
    4. Notion de multi-flot
3. Architecture multicores



Motivation

- Problématique:
  - Exploitation plus efficace des composants de l'architecture,
  - Augmentation du débit des instructions

- Solution :

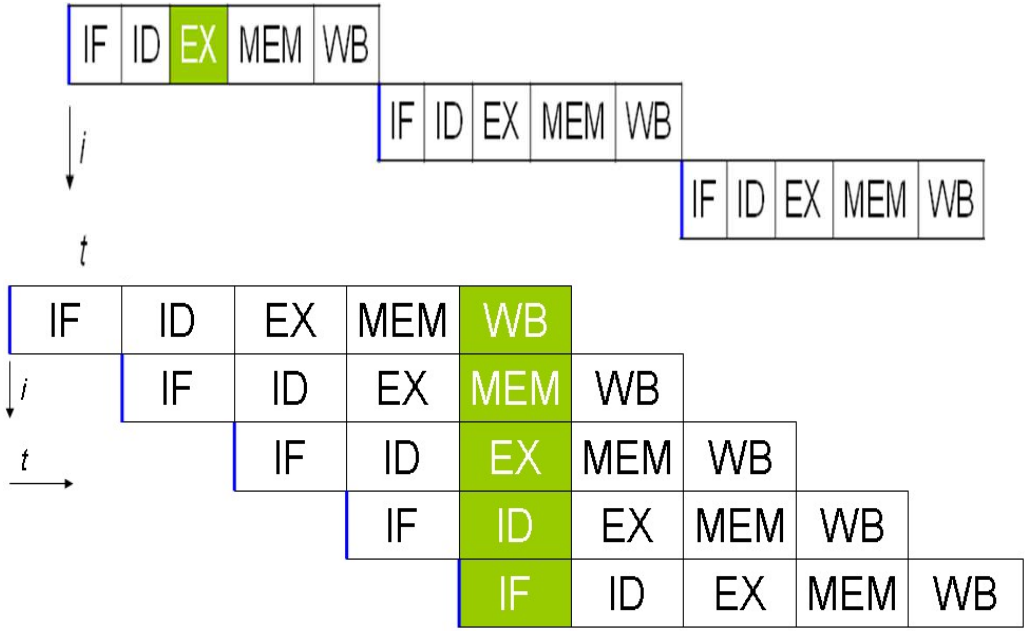
  on segmente les étapes (composants) en sous-étapes (sous-composants) : Augmentation de la vitesse de fonctionnement (F),

⇒ Limite physique : échauffement thermique

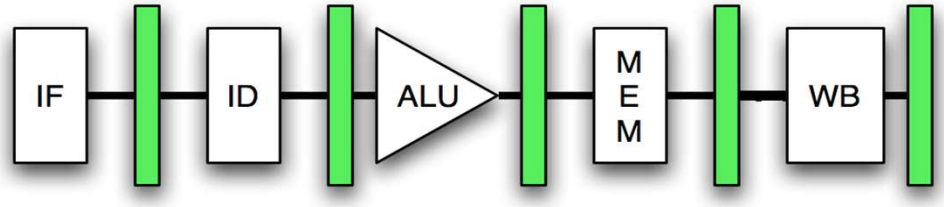
Notion de pipeline

Pour exécuter une instruction, il faut :

- IF : Instruction Fetch
- DEC : Decodage
- Ex : Execution
- Mem : Memory access (load, store)
- WB : Write Back (sauvegarde du résultat dans un reg ou dans la mémoire – move ou load)



- Processeur non pipeline : l'exécution de chaque instruction nécessite 5 cycles d'horloge
- Processeur pipeline : 5 instructions sont en cours et 1 instruction se termine à chaque cycle d'horloge.

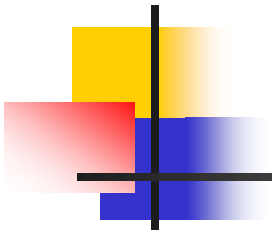


Exemple : « Si (I-J)=0, Suite »

Instr.	t	t+1	t+2	t+3	t+4	t+5	t+6	t+8	t+9
MOV AX,I	IF	DEC	EX	MEM	WB				
MOV BX,J		IF	DEC	EX	MEM	WB			
SUB AX,BX			IF	DEC	EX	MEM	WB		
CMP AX,$0				IF	DEC	EX	MEM	WB	
JE Suite					IF	DEC	EX	MEM	WB

Ex1 : Donner le speedup théorique entre un processeur non pipeline et pipeline en fonction du nombre d'étage de pipeline.

Ex 2 : Calculer le speedup entre un processeur qui exécuterai une instruction en 5ns et un processeur pipeline à 5 étages dont la fréquence de travail est de 1Ghz.



Les problèmes liés au pipeline

Trois types (pipeline stall – gel du pipeline) :

- Aléa Structurel : demandes d'accès à un même composant durant le même cycle
- Aléa de donnée : Dépendance de donnée entre deux instructions
- Aléa de contrôle : Aléa de branchement
  - Si *test* alors … sinon … Résultat du test
    - Vrai : exécution optimale
    - Faux : annule toutes les opérations commencées
  - Exception

Aléa de donnée (Data hazard)

R1 mis à jour

ADD R1<- R1 + 1
ADD R0 <- R1 + R2

Notion de Read After write

Instruction	**T**	**T+1**	**T+2**	**T+3**	T+4	**T+5**
ADD R1<- R1 + 1	IF	DEC	EX	MEM	WB	
ADD R0 <- R1 + R2		IF	DEC	EX	MEM	WB

Instruction	**T**	**T+1**	**T+2**	**T+3**	T+4	**T+5**
ADD R1<- R1 + 1	IF	DEC	EX	MEM	WB	
ADD R0 <- R1 + R2		IF	DEC	**X**	EX	MEM …

R1 + R2 ?

ADD
NOP
ADD ..

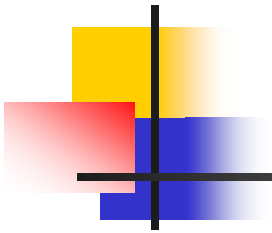
Aléa de contrôle (Control hazard)

```
IF A>B Then
        Y(i) = Y(i) + X(i).H(i)
   ELSE
        N=B-A;
        FOR (n=1 TO 16)
        {
        Zn = Zn-1 + Xn.Hn
        }
        END FOR;
END IF;
```

Donner l'ordre d'exécution des instructions.

Déterminer la pénalité temporelle due aux aléas du pipeline.

En déduire le temps de réponse.



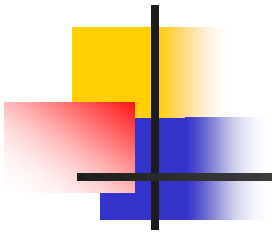
Remède - Problème lié au contrôle

Prédiction de branchement => prédire l'adresse de branchement qui est stockée dans une table.

PC <= PC + 1 ou

PC <=adresse de branchement (on parle alors de branchement pris)

- **Prédiction non conditionnelle**
  - Appel de fonction => Saut non conditionnel (JUMP)
- **Prédiction conditionnelle :**
  - Prédiction statique :
    - Toujours pris => pour les boucles, aléa à la sortie de la boucle
    - Analyse à la compilation



Remède - Problème lié au contrôle

- Prédiction dynamique :
  - Pour les boucles :

    Mise à jour de l'adresse de la table après obtention de la valeur de la condition

  - Pour les IF, …

    Statistiques sur les branchements antérieur (mauvais rendement).

Quelques chiffres

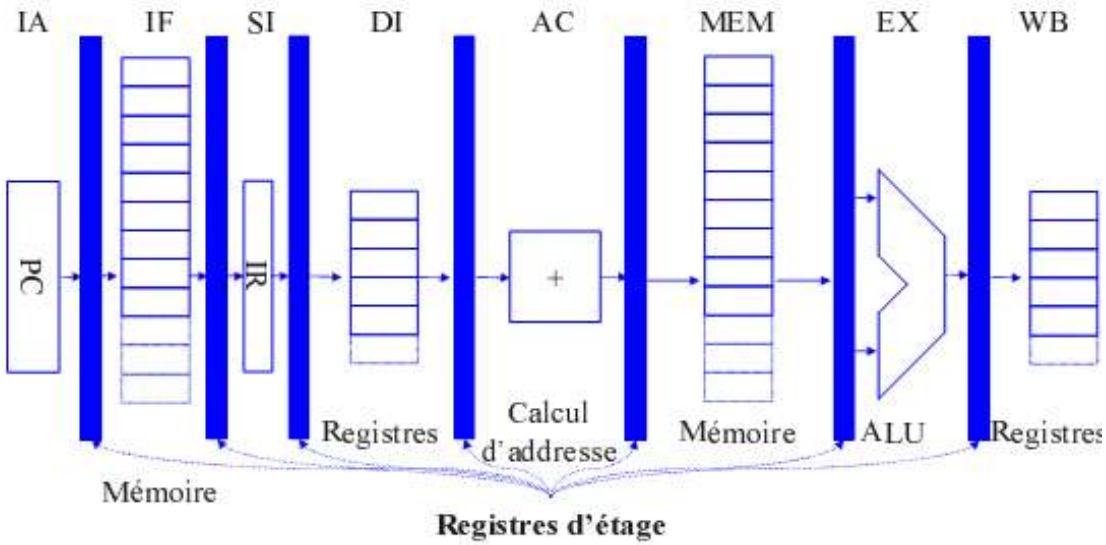
- En moyenne, 1 instruction sur 5 est un branchement.
- Sur les pipelines actuels, le coût d'un aléa de contrôle est de 5 à 10 cycles.

- 8 cycles entre le fetch et la résolution du branchement
- 4 cycles pour réinitialiser le pipeline

⇒ 12 cycles de pénalité.

- En moyenne 1 branchement toutes les 5 instructions.
- 1 instruction / cycle (1000 instructions) :
  - 50% mauvaises prédictions:
    1000 * (0,8*1 + 0,2*(0,5*1 + 0,5*12)) = 2100 cycles
  - 20% mauvaises prédictions:
    1000 * (0,8*1 + 0,2*(0,8*1 + 0,2*12)) = 1440 cycles
  - 5% mauvaises prédictions:
    1000 * (0,8*1 + 0,2*(0,95*1 + 0,05*12)) = 1110 cycles
- 5 instructions / cycle (1000 instructions) :
  - 50% mauvaises prédictions:
    200 * (0,5*1 + 0,5*12) = 1300 cycles
  - 20% mauvaises prédictions:
    200 * (0,8*1 + 0,2*12) = 640 cycles
  - 5% mauvaises prédictions:
    200 * (0,95*1 + 0,05*12) = 310 cycles

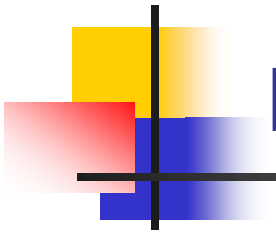
Exercices



On donne ci-dessous un exemple de pipeline. Déterminer le speedup par rapport à une machine non pipeline et pipeline (cas idéal).

Indiquer les pertes en fonction des différents types de rupture de pipeline.

Calculer le temps d'exécution pour une application MPEG2 (calcul de Err) sans prédiction de branchement, avec prédiction de branchement statique « toujours pris » et dynamique.



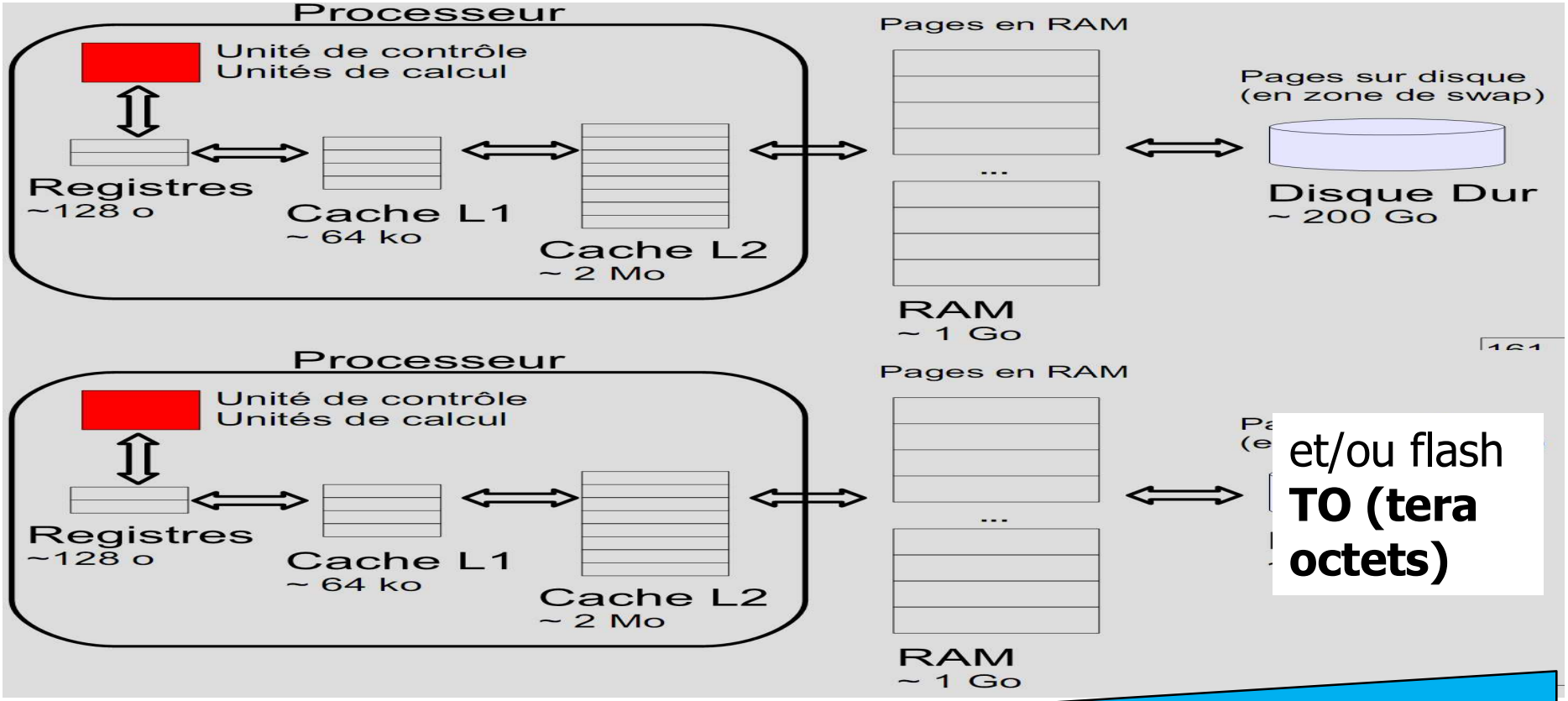
PLAN

1. Introduction & Paramètres importants
2. Architecture d'un core :
    1. Architecture pipeline
    2. Architecture du système mémoire

    Mémoire cache, Mémoire principale

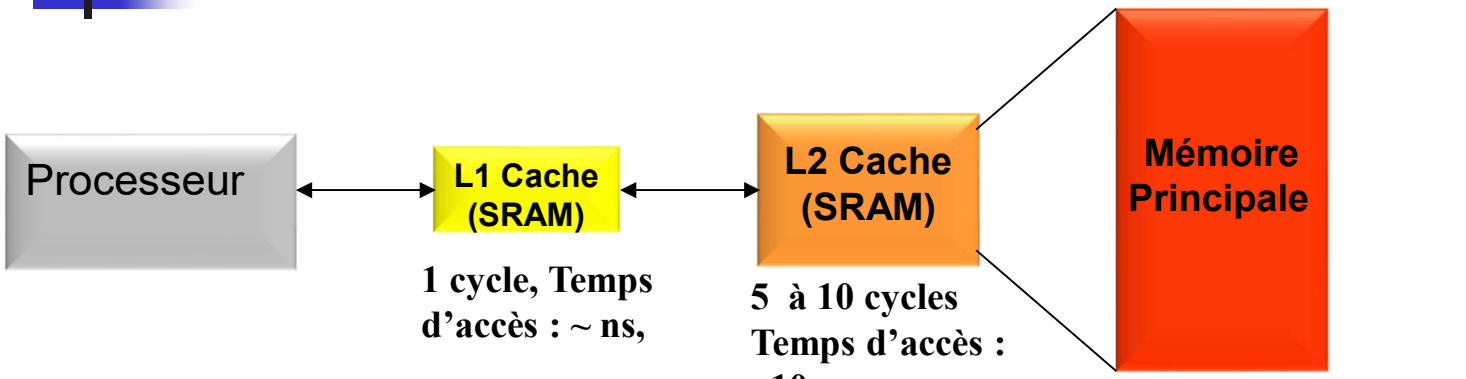
    3. Architecture Superscalaire et Architecture VLIW
3. Architecture multicores

Architecture du système mémoire



Temps d'accès à la mémoire

Architecture du système mémoire et consommation



Cellule SRAM : 6Trs

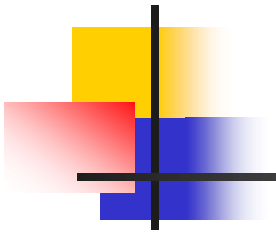
Cellule DRAM : 2 Trs

Mémoire cache L1

- En général, même fréquence que le CPU
- Taille réduite : de 16 Ko, 32Ko, 64Ko-128Ko
- Aucun contrôle sur le contenu du cache

  => Un programme ne manipule jamais directement le cache

  => Il se remplit au fur et à mesure de l'exécution des programmes



Cache et propriétés

Le cache exploite deux propriétés :

- Localité temporelle : une adresse A référencée à l'instant t a de fortes chances d'être référencées dans un court intervalle de temps.

  Exemple : opération accumulation
- Localité spatiale : si une adresse A est référencée à l'instant t, il y a de forte chance qu'une adresse voisine de A soit référencée dans un court intervalle de temps.

  Exemple : tableau, matrices, …
- Deux types de cache : données, instructions

La majorité des programmes possèdent ces propriétés.
Si ce n'est pas le cas, les performances sont dégradées.

Localité des données et des instructions

Exemple :

```
For (i=0, i<N, i++)
        For (j=0, j<N, j++))
        Diff_pixel= I_i[i][j] –I_i-1[i][j];
        Diff_image = Diff_image + Diff_pixel ;
```

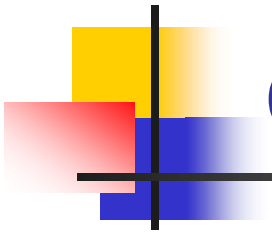
Boucle => Instructions

- **réutilisation des instructions : Localité temporelle**
- **instructions consécutives en mémoire : localité spatiale**

Boucle : Données

I[i][j] : propriété de localité spatiale

Diff_Image: propriété de localité temporelle,



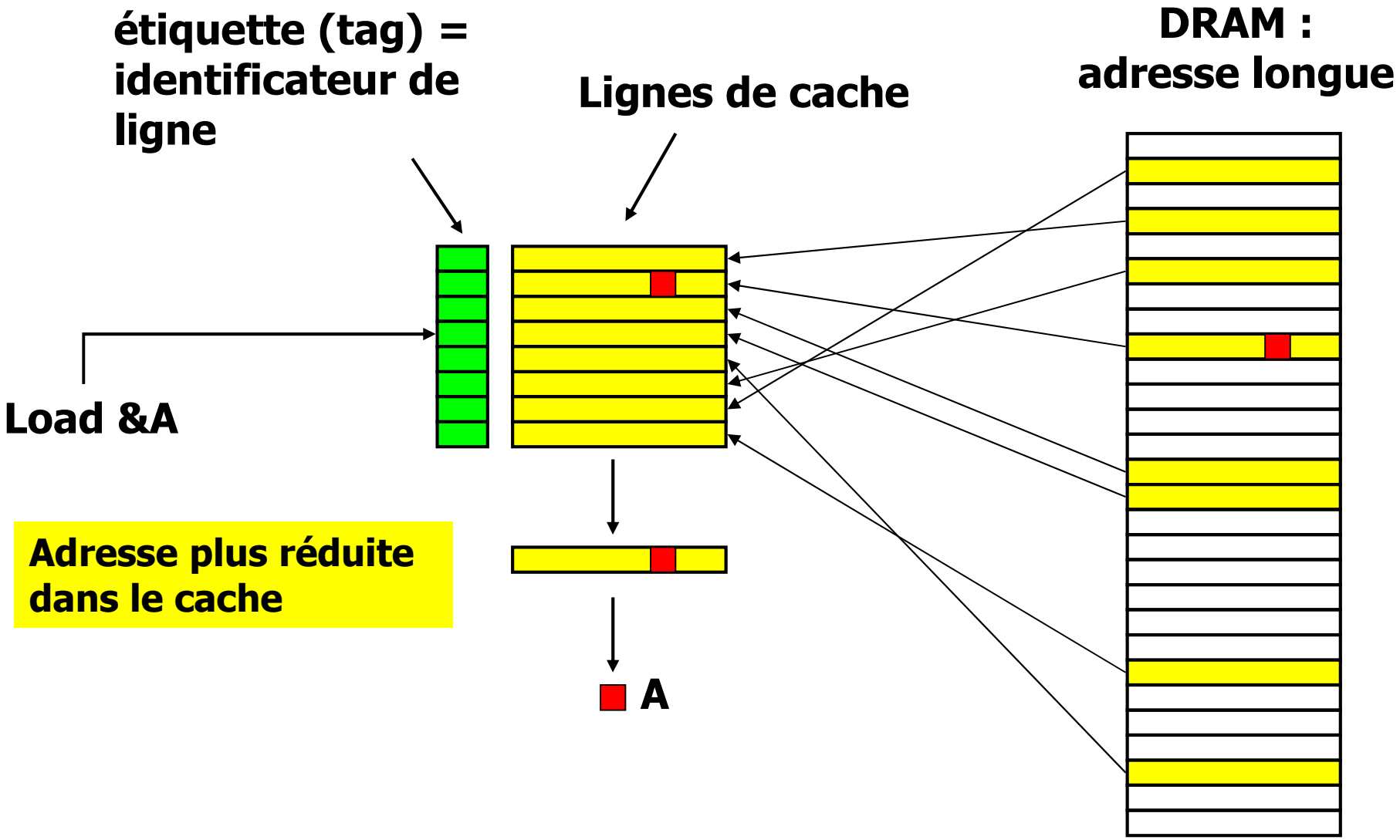
Cache et pénalités

Considérons un programme ayant un temps d'exécution de 100 cycles.

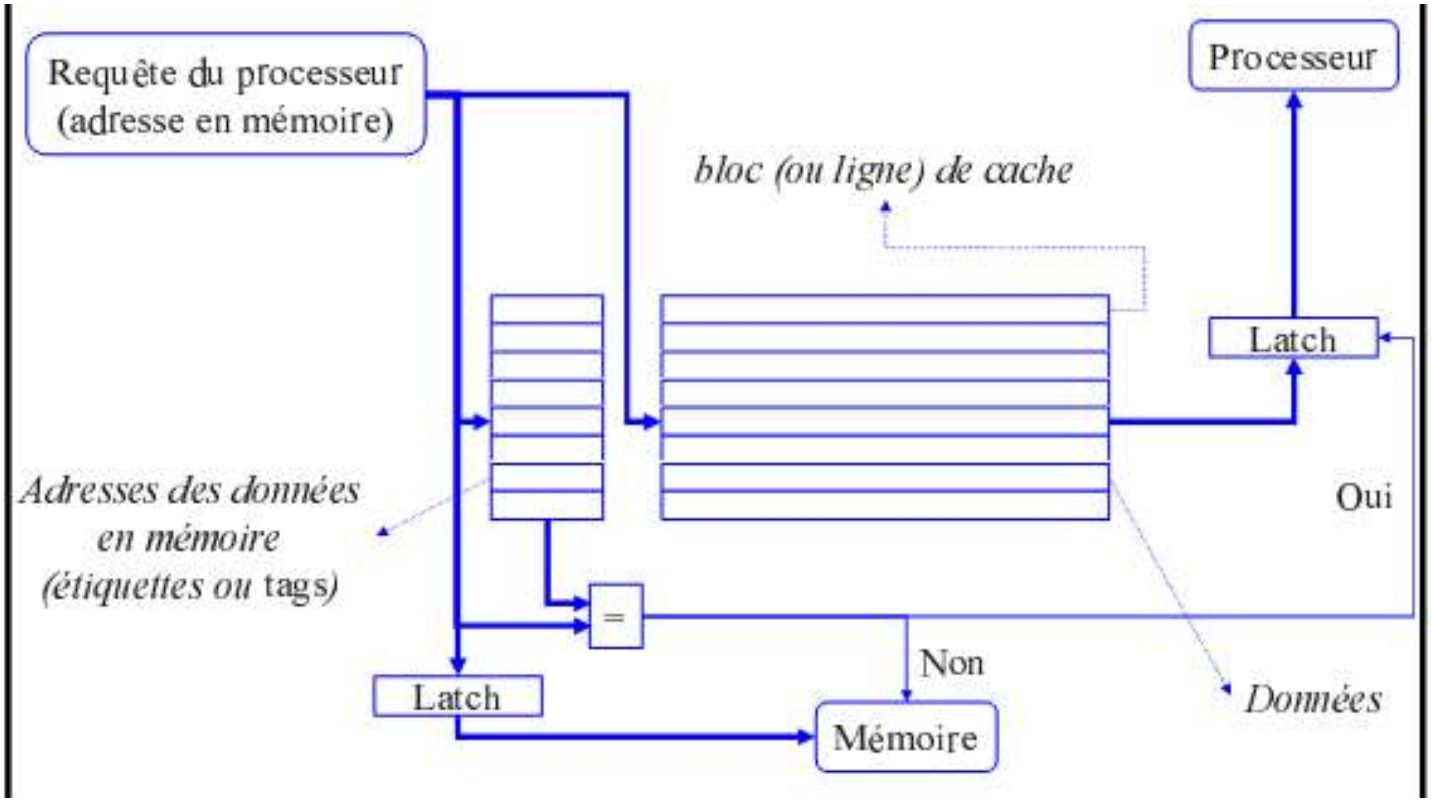
Que devient ce temps d'exécution si :

- 50% des instructions sont des instructions mémoire.
- le taux de cache miss (miss rate) est de 20%,
- l'accès à la mémoire externe (DRAM) nécessite 10 cycles d'horloge.

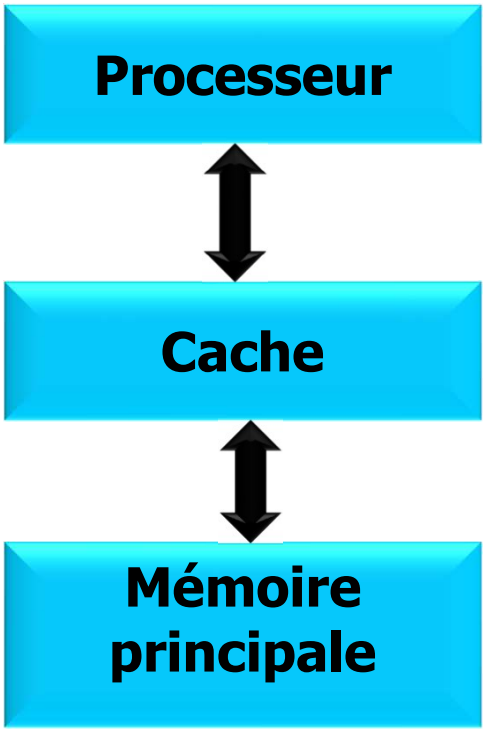
Architecture d'un cache : gestion des adresses



Accès aux données ou instructions



Architecture du système mémoire : exemple



Cache Hit : la donnée est dans le cache
Cache miss : la donnée n'est pas dans le cache

```
For (i=0;i<512;i++)
        For (j=0;j<512; j++)
                Diff=Diff + Ip(i,j)-Ip-1(i,j);
```

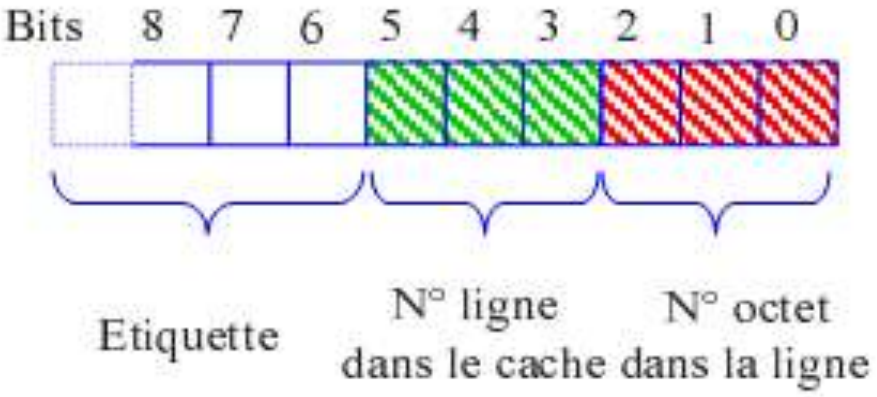
Déterminer le nombre min et max de cache hit et de cache miss.

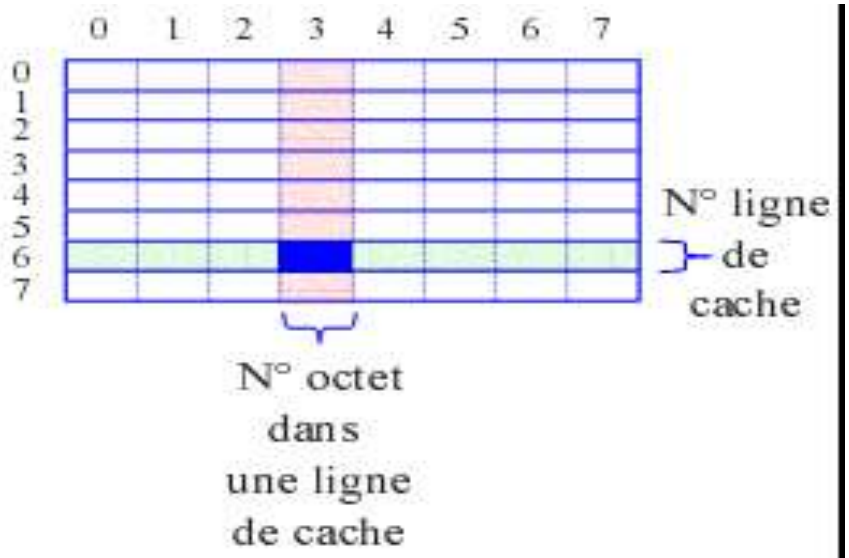
Reprendre cette question pour le calcul de l'histogramme d'une image :

```
Integer Histo[256];
For (i=0;i<512;i++)
        For (j=0;j<512; j++)
                level = Image[i][j];
                Histo[level]=Histo[level]+1;
```

Placement des données

- Le placement des données est géré par le matériel : le fonctionnement du cache est transparent pour l'utilisateur.
- 1 octet = 1 adresse, les données sont chargées dans le cache par bloc (ligne de cache).
- Le placement d'une donnée est déterminé par :
  - Numéro de ligne,
  - Numéro d'octet dans la ligne.



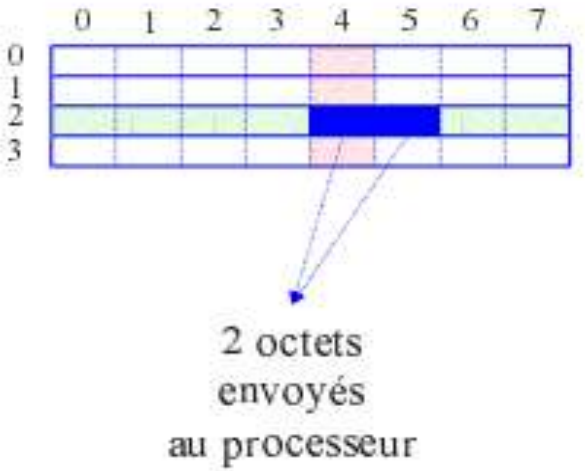


Lecture d'une donnée

- Cs : taille du cache en octet
- Ls : taille de la ligne (en termes d'octet)
- N° octet : $Log_2(L_s)$ poids faible de l'adresse
- N° ligne : $Log_2(C_s/L_s)$ poids forts de l'adresse.

Exemple : Cs = 32 octets et Ls =8 octets

- Adresse demandée (16 bits) : 0110 0100 0101 0100
- N° de ligne : 10
- N° Octet dans la ligne 100

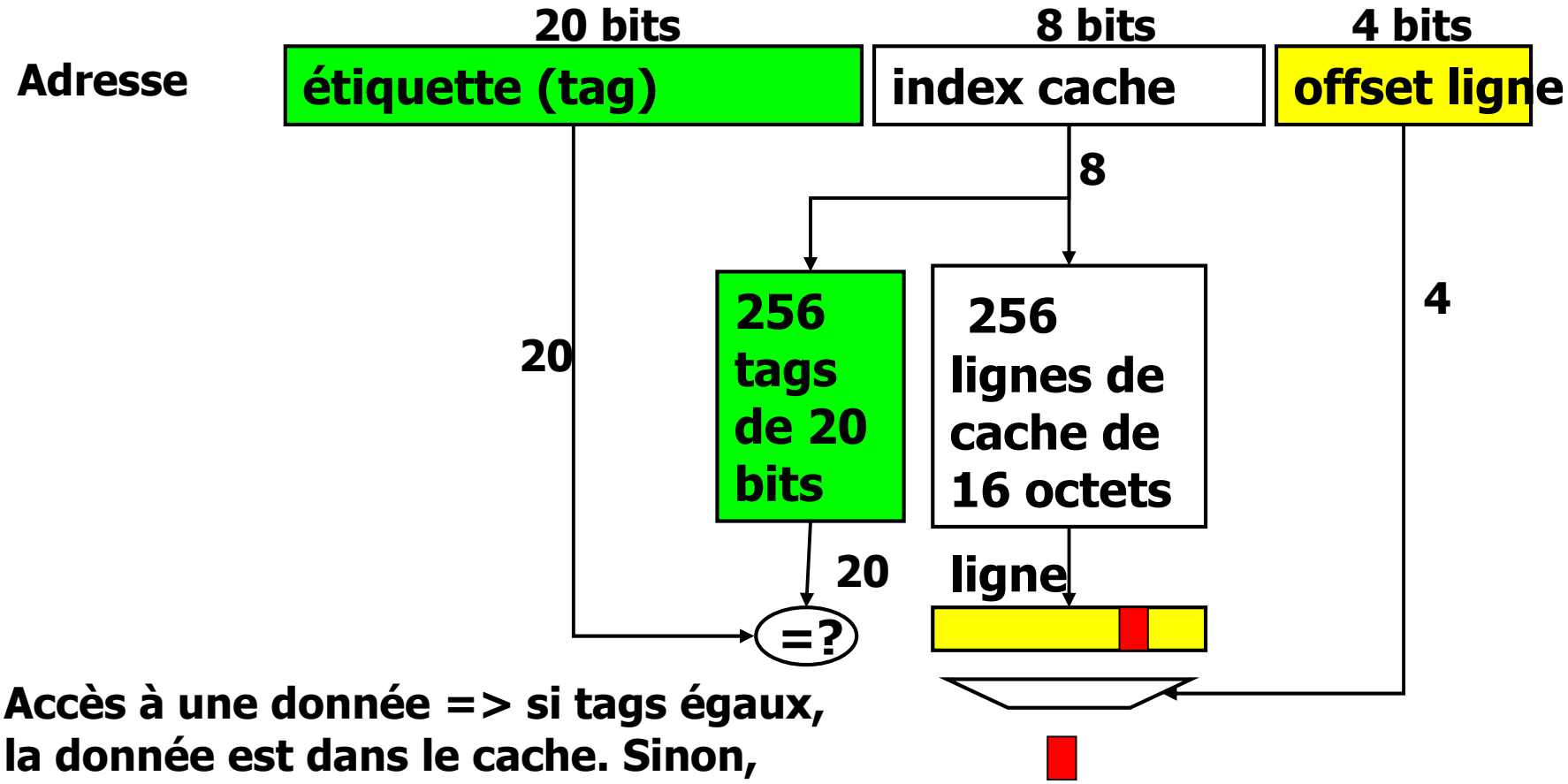


Une requête peut avoir une taille variable :

Requête = adresse + nombre d'octets

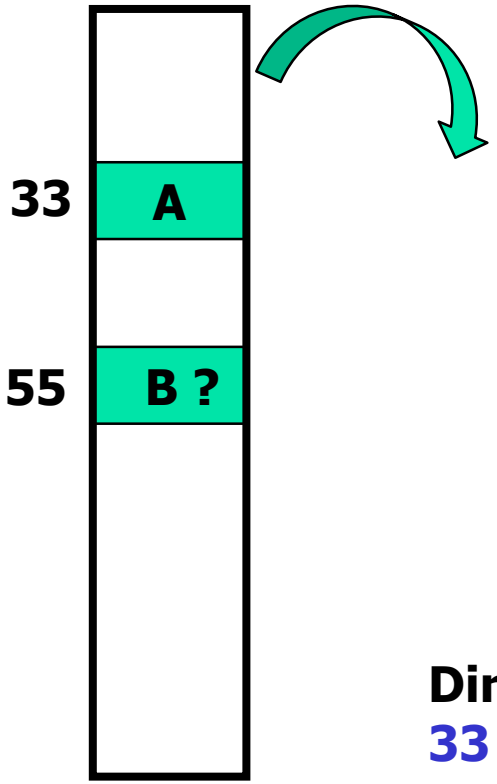
Exemple de cache

Capacité : 4 Ko, lignes de 16 octets, adresses 32 bits

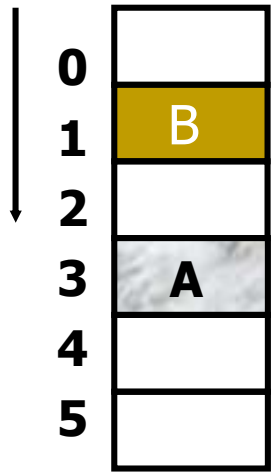


Accès à une donnée => si tags égaux, la donnée est dans le cache. Sinon, défaut de cache.

Gestion des adresses – cache non associatif



Mémoire principale
(main memory)



Direct Mapped Cache
33 mod 6 = 3

Si ligne de cache = une donnée

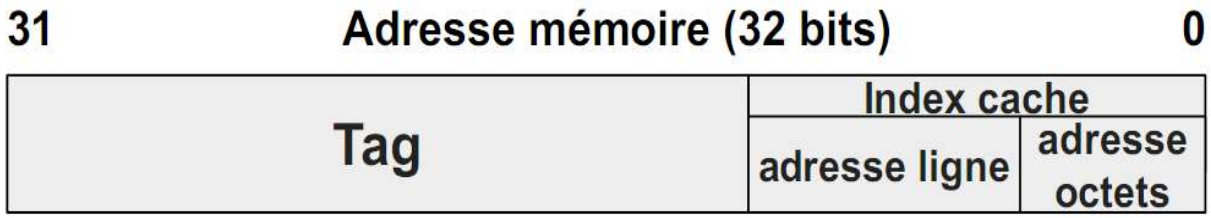
Adresse Cache =

(Addresse) mod (taille du cache)

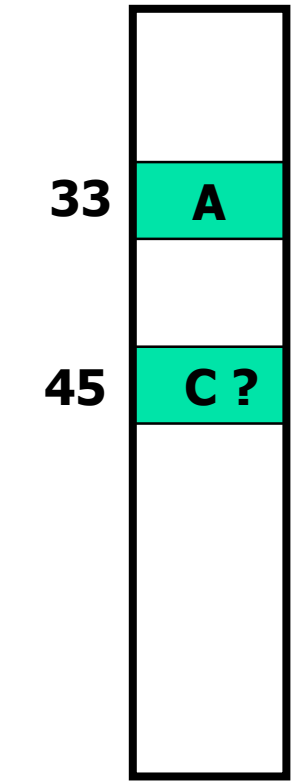
Si la ligne du cache contient M mots :

Ligne de cache =

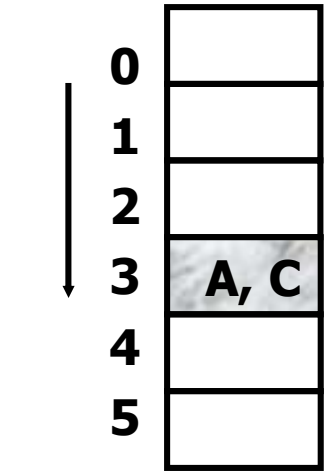
(Adresse mod (taille du cache))/M



Gestion des adresses – cache associatif

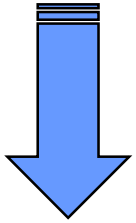


Mémoire principale

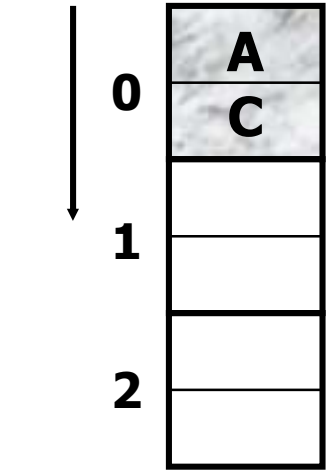


Direct Mapped Cache

33 mod 6 = 3

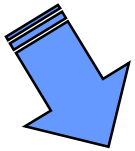


Défaut de cache

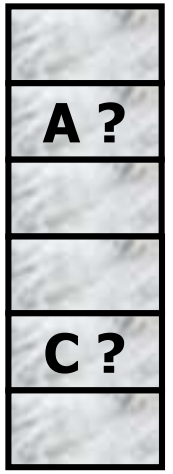


Associativité : 2

33 mod 3 = 0



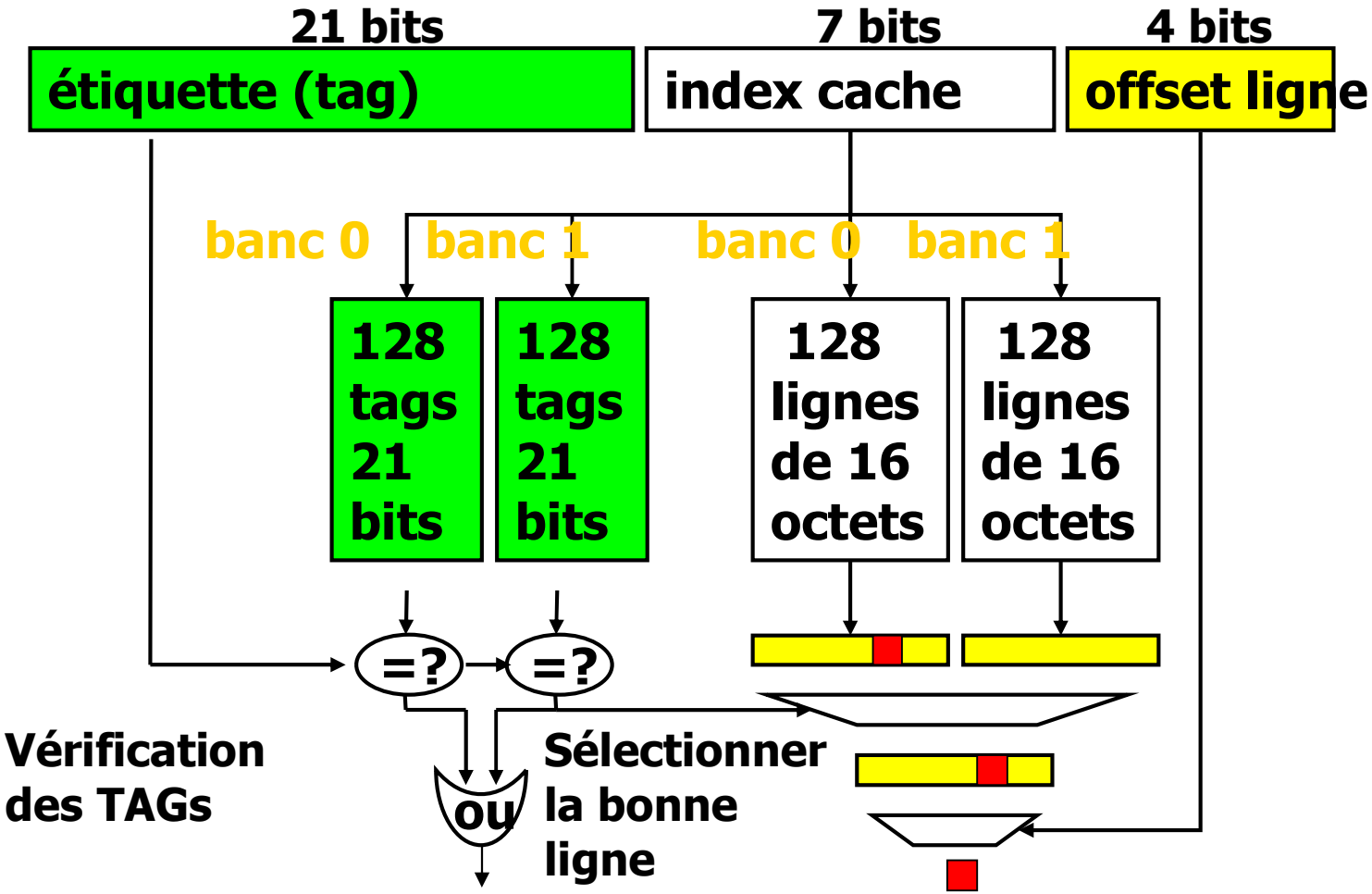
taille du cache/degré d'associativité



Fully associative cache

Exemple de cache associatif - 4Ko

Associativité : 2, lignes de cache : 16 octets, adresse : 32 bits



Défauts de cache

Trois catégories :

- **Obligatoires : premier accès à une ligne de cache (ou donnée) qui n'ai pas dans le cache.**
- **De capacité : le cache ne peut pas contenir tous les blocs nécessaires. Au cours d'exécution d'un programme, des lignes de cache seront écrasées puis rappelées plus tard.**
- **De conflit : Une ligne de cache est écartée (puis rappelée) car elle est placée à des adresses similaires à d'autres, lignes de cache utilisées par un programme.**

Obligatoire :

Premier accès aux éléments d'un tableau: A[j][i] = B[j][i] * C[j][i];

Capacité :

Cache : 16KB , Image : 1920*1080,

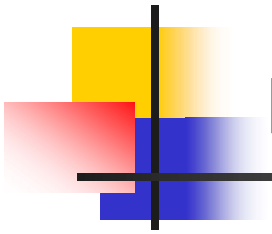
Traitement : Iprevious, Iin, Iout

Conflit:

Cache : 16KB

Tableau A : ADDR =0

Tableau B : ADDR =16384



Défauts de cache et pénalités sur le CPUtime

Exercice : On donne le programme suivant :

```
Char A[64][64], B[64][64], C[64][64]; /* char = 1 octet */
for (i=0; i<64; i++)  -- LOOP2
        {for (j=0; j<64; j++)   -- LOOP1
                {A[i][j] = B[i][j] * C[i][j]; }}
```

Donner l'expression du temps de réponse:

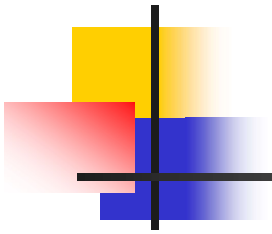
- dans le cas idéal (pas de défaut de cache)

- en tenant compte du nombre de défaut de cache (cache miss)

en fonction du nombre de défaut de cache (précisez le type de défaut) pour une mémoire cache : direct mapped, 4Ko, taille de la ligne de cache = 64 Octets (cas fréquent).

Caractéristique du Processeur :

Fréquence CPU= 1 Ghz, Temps d'accès à la mémoire externe = 10 ns.



Les mémoires cache - associativité

Pour les caches associatifs, trois principales stratégies de mise en mémoire :

- LRU (last recently used) :

La ligne de cache la plus anciennement utilisée est choisie (complexe à mettre en œuvre).

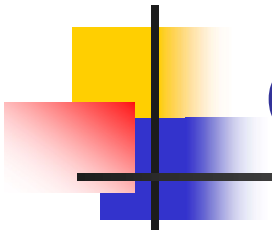
- FIFO la ligne de cache :

La ligne de cache utilisée au plus tôt est choisie.

- Au hasard :

La ligne de cache est choisie aléatoirement (stratégie plus facile à réaliser).

La LRU diminue plus fortement le taux d'échec que les autres stratégies.



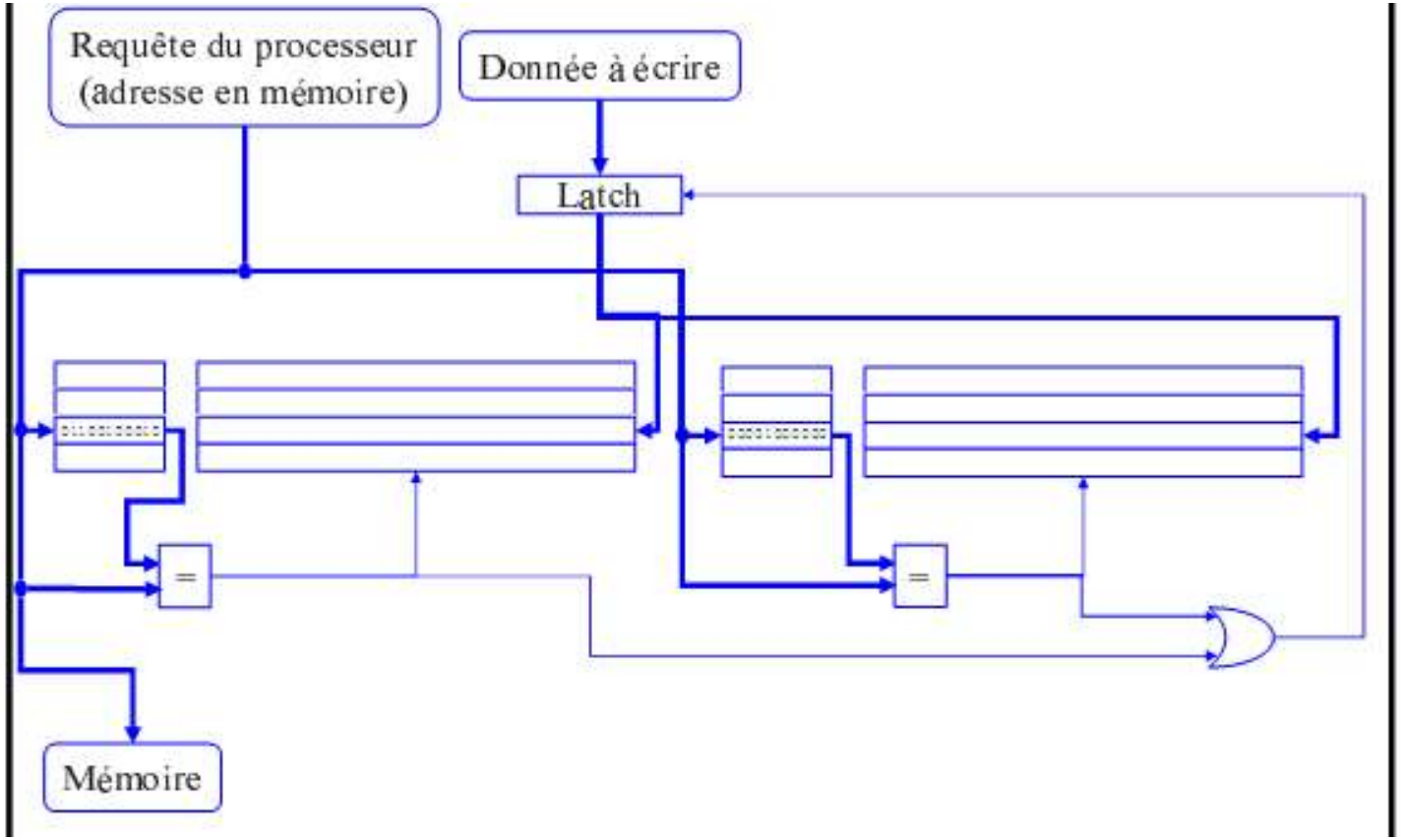
Gestion des caches en écriture

- **Write through :**

  la donnée est écrite à la fois dans le cache et dans la mémoire principale. La mémoire principale et le cache ont à tout moment une valeur identique.
- **Write back :**

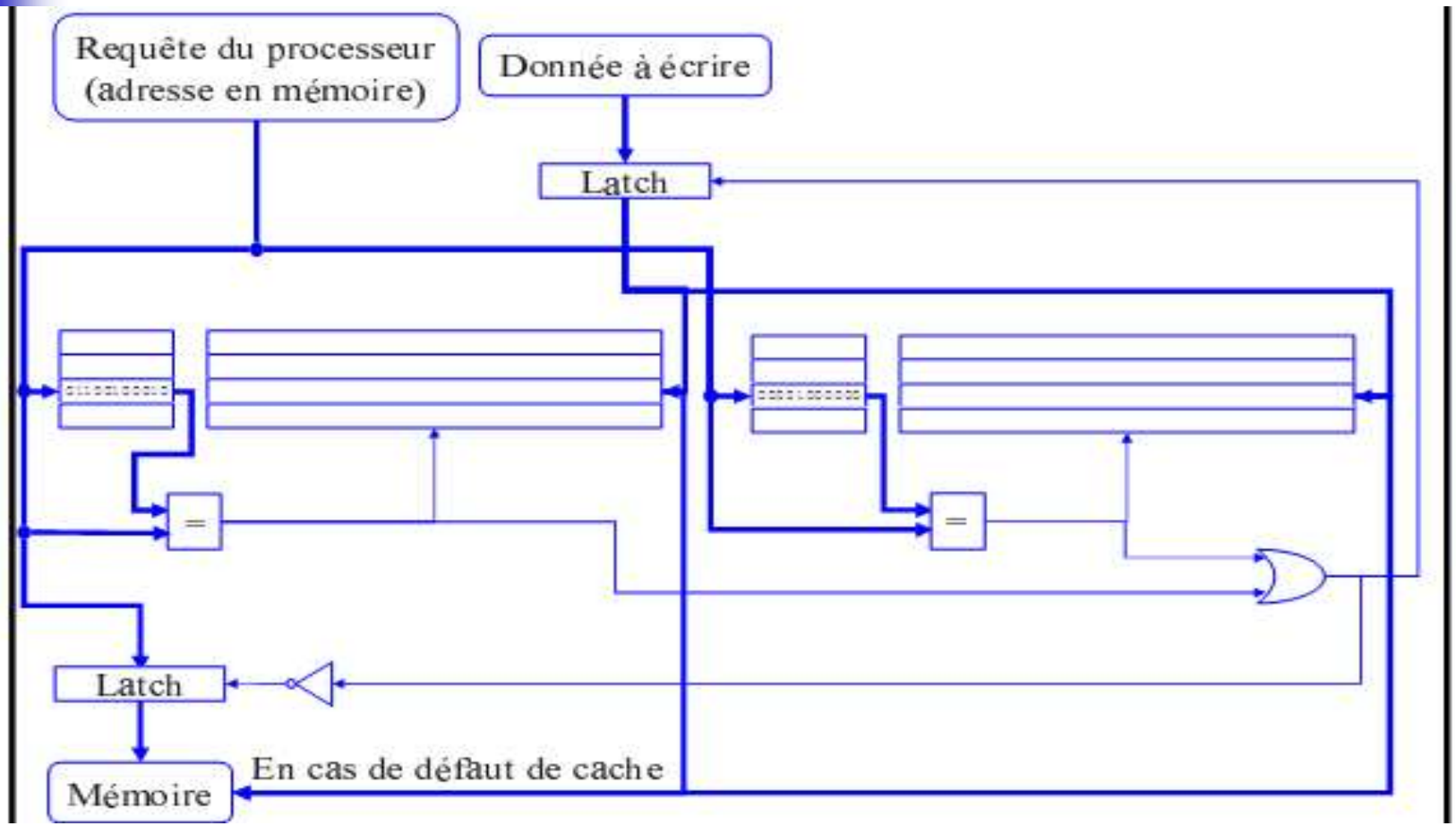
  la donnée est écrite dans la mémoire principale lorsque la ligne disparaît du cache. Un bit nommé *dirty bit est activé quand une donnée de la ligne* a été modifiée.
- **write buffer :**
  - données à écrire stockées temporairement,
  - écriture différée pour ne pas bloquer le processeur

Ecriture d'une donnée – Gestion Write through

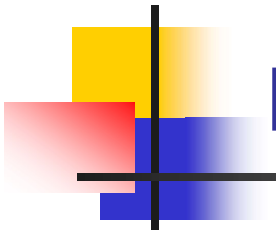


La donnée est écrite dans le cache et dans la mémoire (cohérence de cache)

Ecriture d'une donnée – Gestion Write back



La donnée est écrite dans le cache. Le bloc modifié est écrit en mémoire quand ce bloc disparait du cache. Le bloc est marqué par 1 bit pour indiquer si il a été modifié ou non.



Exercice

Ex 1 : Considérons un cache « fully associative ». Au départ, les adresses 100 et 200 sont le cache. La séquence d'accès est :

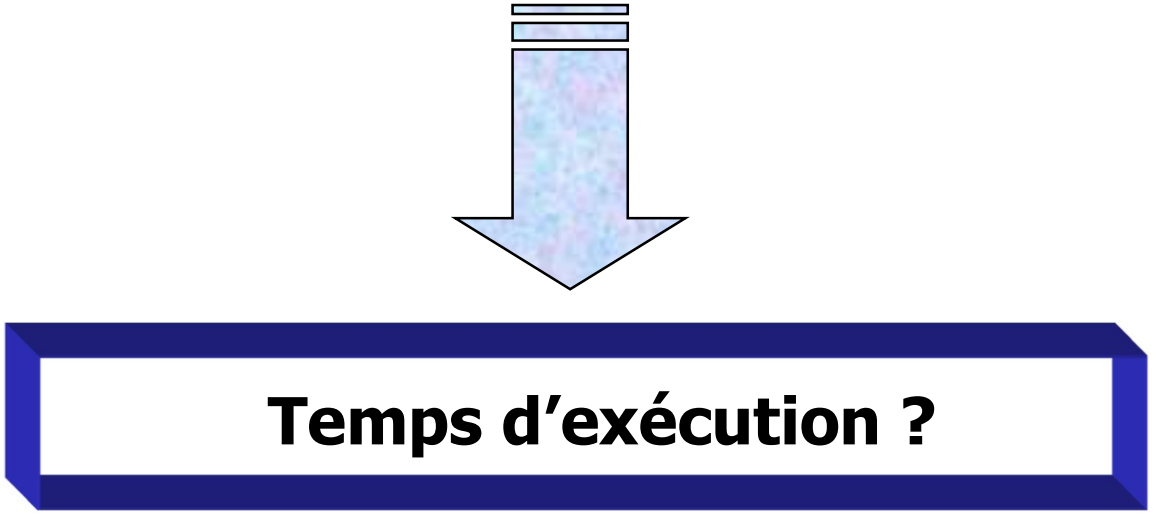
- Write mem[100]
- Read mem[100]
- Write mem[100]
- Read mem[200]
- Write mem[200]
- Write mem[100]

Déterminer le nombre de hit et de miss pour les deux politiques d'écriture en mémoire principale.

Remède associé aux défauts de cache

Trois catégories :

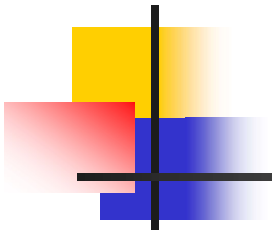
- ⇒ **Echec obligatoire : ligne de cache plus importante,**
- ⇒ **Echec de capacité : cache de plus grande capacité,**
- ⇒ **Echec de conflits : associativité plus élevés.**



Défaut de cache – Exemple application audio

Cache size (Ko)	Associativité	Miss rate %
4	1	9,8
4	2	7,6
4	4	7,1
4	8	7,1
8	1	6,8
8	2	4,9
8	4	4,4
8	8	4,4

Cache size (Ko)	Associativité	Miss rate %
16	1	4,9
16	2	4,1
16	4	4,1
16	8	4,1
32	1	4,2
32	2	3,8
32	4	3,7
32	8	3,7



Défauts de cache – Exercice

Exemple d'étude de configurations du cache:

Clock cycle time$_{2\text{-way}}$ = 1,36x Clock cycle time$_{1\text{-way}}$
Clock cycle time$_{4\text{-way}}$ = 1,44x Clock cycle time$_{1\text{-way}}$
Clock cycle time$_{8\text{-way}}$ = 1,52x Clock cycle time$_{1\text{-way}}$

Hit : 1 cycle, Miss : 10 cycles d'horloge (accès au L2).

A partie de la table donnée précédemment, quelle est la meilleure configuration du cache pour l'application considérée?

Estimation du temps d'exécution

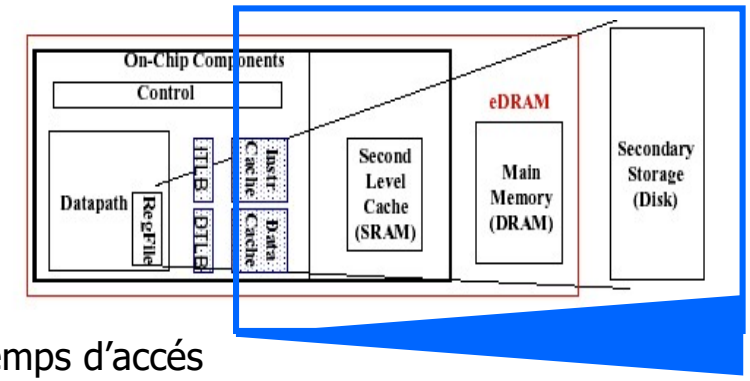
$$\text{CPU time} = \Sigma_i \, \text{CPUtime}_i \, N_i + \Sigma_k \, E_k$$

Avec CPUtime_i : le temps d'exécution pour l'instruction i,

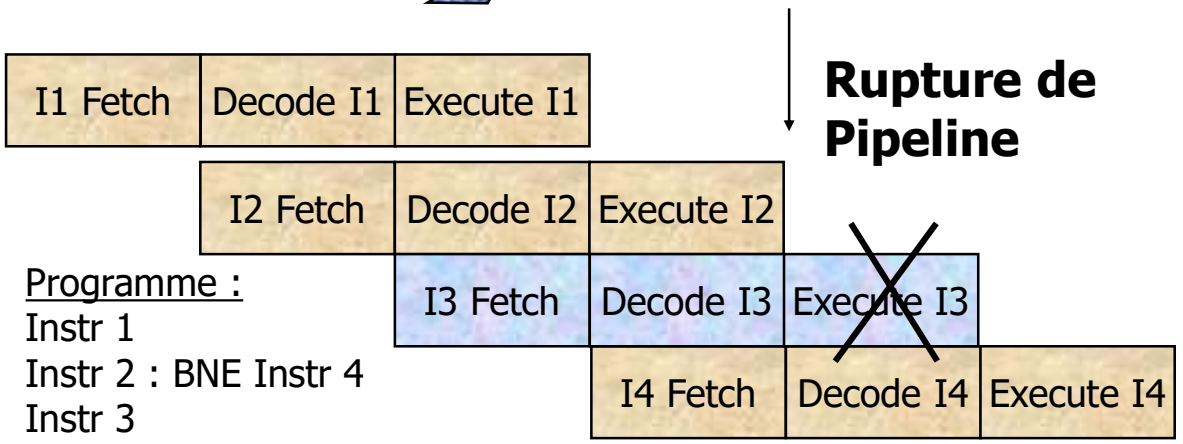
Ni : nombre d'instruction de type i,

Ek : pénalités temporelles dues aux défauts de cache, et aux ruptures de pipeline.

Les données (les instructions) ne sont pas dans le cache. Le processor attend : Eproc ~ Enop



Temps d'accés

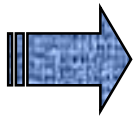


Programme :
Instr 1
Instr 2 : BNE Instr 4
Instr 3
Instr 4

Défaut de cache – Remède : transformation de code

```
for(i=0;i<N;i++)
  for(j=0;j<N;j++)
          A[j][i] = A[j][i] +1;
```

Loop Interchange

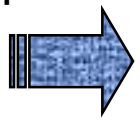


```
for(j=0;j<N;j++)
  for(i=0;i<N;i++)
          A[j][i] = A[j][i] +1;
```

Hypothèse : les données sont stockées par ligne, A[j][i] et A[j][i+1] sont à des adresses consécutives.

```
for(i=0;i<N;i++)
  B[i]=A[i] +1;
for(i=0;i<N ;i++)
   C[i]=A[i] * 3;
```

Loop Fusion

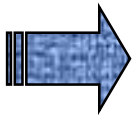


```
for(i=0;i<N;i++)
          B[i]=A[i] +1;
          C[i]=A[i] * 3;
```

Cette transformation réduit le nombre de défauts de cache. Si A peut être affectée à un registre => une seule lecture mémoire.

```
for(i=0;i<N;i++)
  B[i]=A[i] + A[i+1];
```

Loop Unrolling



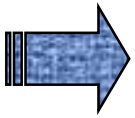
```
for(i=0;i<(N/2)-1;i+2)
  B[i]=A[i] + A[i+1];
  B[i+1]=A[i+1] + A[i+2];
```

Si A[i+1] peut être affectée à un registre => une seule lecture mémoire.
Pb: Augmente la taille du code (cache d'instruction).

Défaut de cache – Remède : transformation de code

Multiplication de matrice

```
for(i=0;i<N;i++)
 for(j=0;j<N ;j++)
  for(k=0;k<N ;k++)
     C[i][j]= C[i][j] + A[i][k] * B[k][i];
```

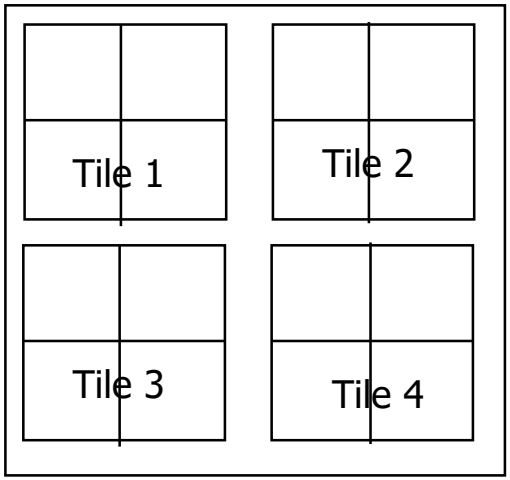


```
for(j'=0;j'<N;j'+T)
 for(k'=0;k'<N ;k'+T)
/* un block de taille T */
  for(i=0;i<N;i++)
    for(j=j';j< min(j'+T-1,N-1);j++)
      for(k=k';k< min(k'+T-1,N-1);k++)
        C[i][j]= C[i][j] + A[i][k] * B[k][i];
```

Loop Tiling Transformation

A, B, C ne rentrent pas dans le cache => décomposition par bloc dont la taille rentre dans le cache (Tile size).

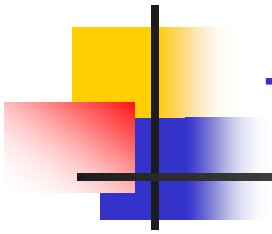
=> Autre solution : modification du calcul d'adresse associé à un code de gray



Tile 1 : A[0][0], A[0][1], A[1][0], A[1][1],

Tile 2 : A[0][2], A[0][3], A[1][2], A[1][3],

…



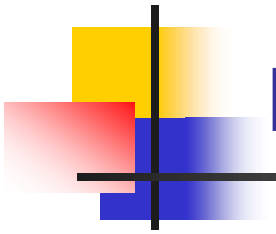
Transformation de code : exercice

Considérons le programme suivant :

```
Char A[64][64], B[64][64], C[64][64], D[64][64], F[64][64];
/* char = 1octet */
for (i=0; i<64; i++)
    {for (j=0; j<64; j++)  {A[j][i] = B[j][i] * C[j][i];}
for (i=0; i<64; i++)
        {for (j=0; j<64; j++)  {D[j][i] = C[j][i] - F[j][i]};}
```

Evaluer le nombre de cache hit et de cache miss. Proposer des transformations de code permettant de réduire le taux de cache miss (miss rate). En déduire le temps d'exécution.

(Cache direct mapped, 8 octets par ligne de cache). On fera l'hypothèse qu'il n'y a pas de conflit d'adresse.



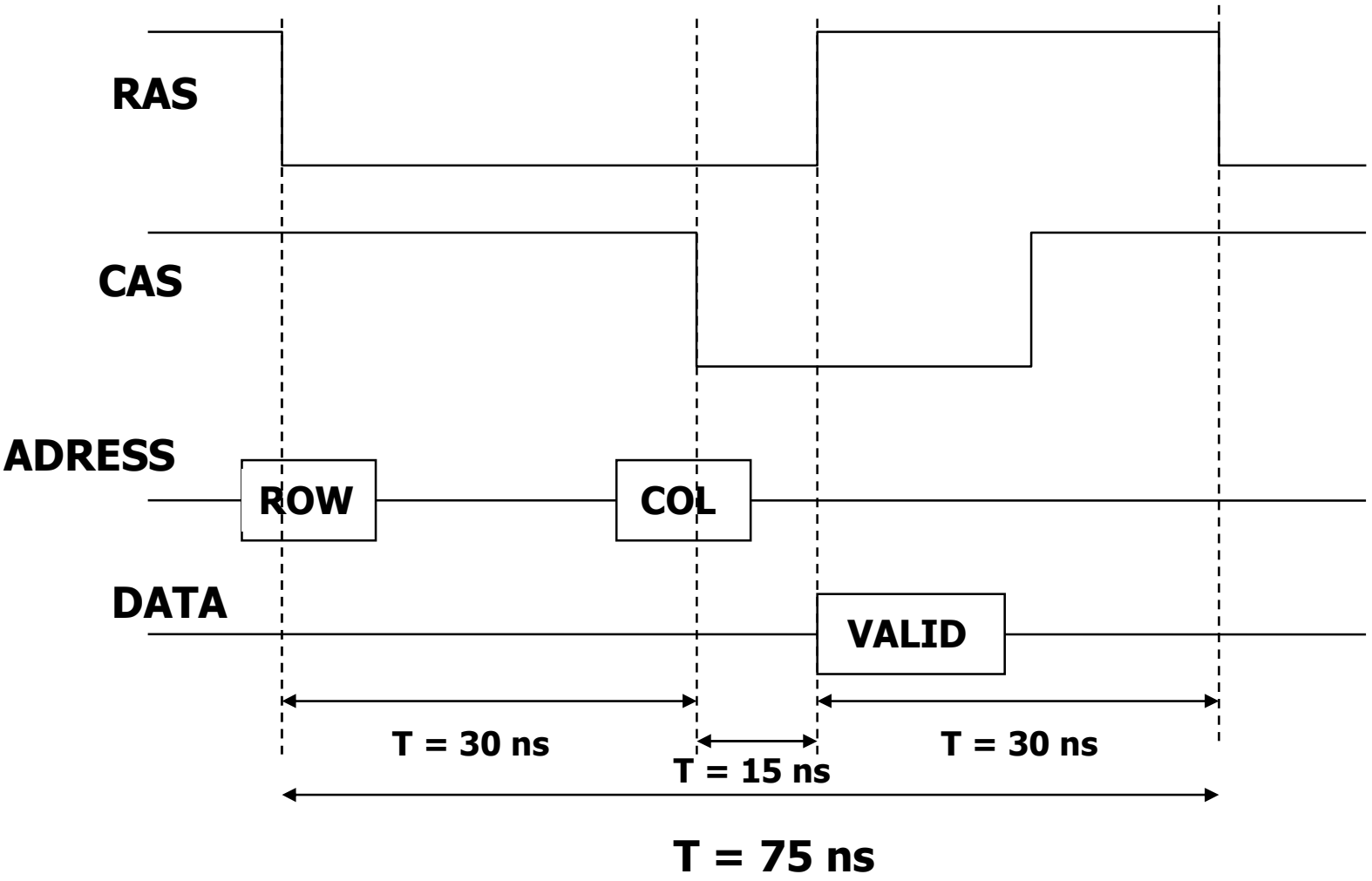
PLAN

1. **Introduction et Paramètres importants**
2. **Architecture d'une core**
    1. Architecture pipeline
    2. Architecture du système mémoire :

        Mémoire cache, **Mémoire principale**

    3. Architecture Superscalaire et VLIW
3. **Architecture multicores**

Fonctionnement des DRAM



Modes :

- READ,

- WRITE,

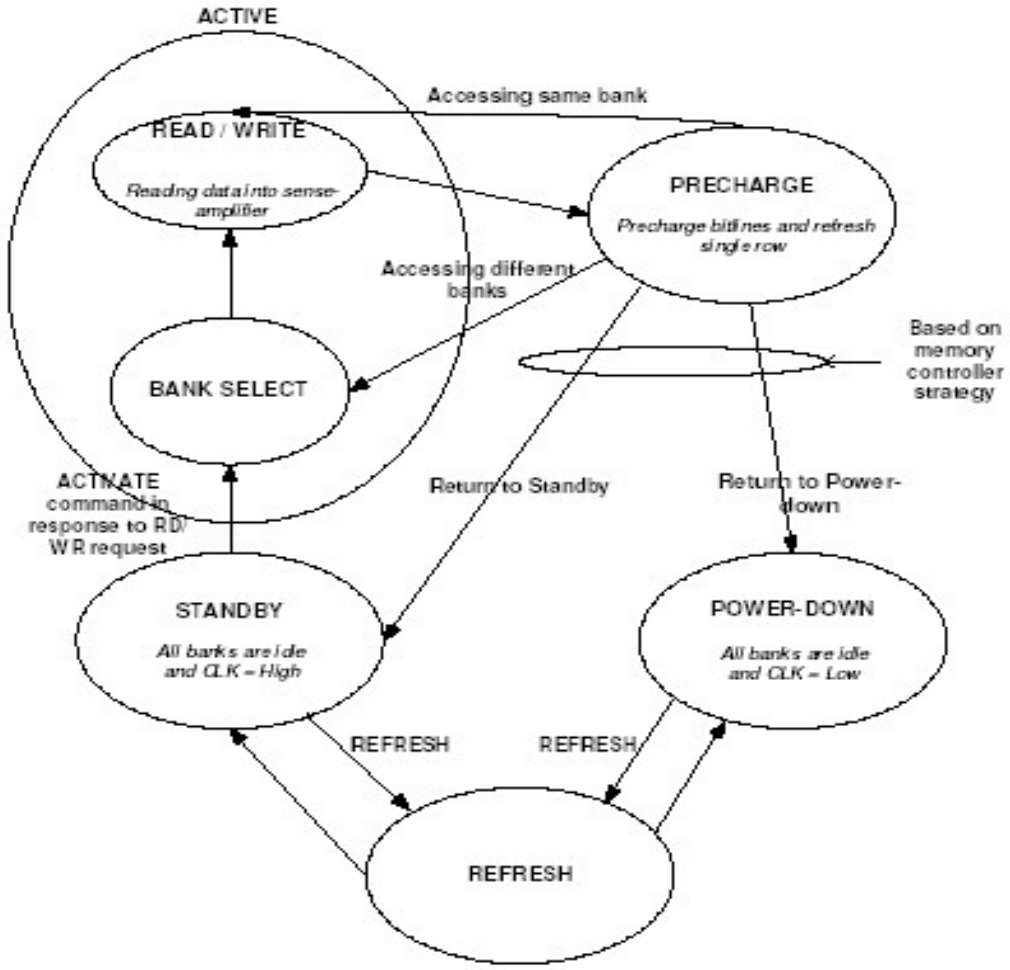
- R-M-W.

Fonctionnement des DRAM : SDRAM

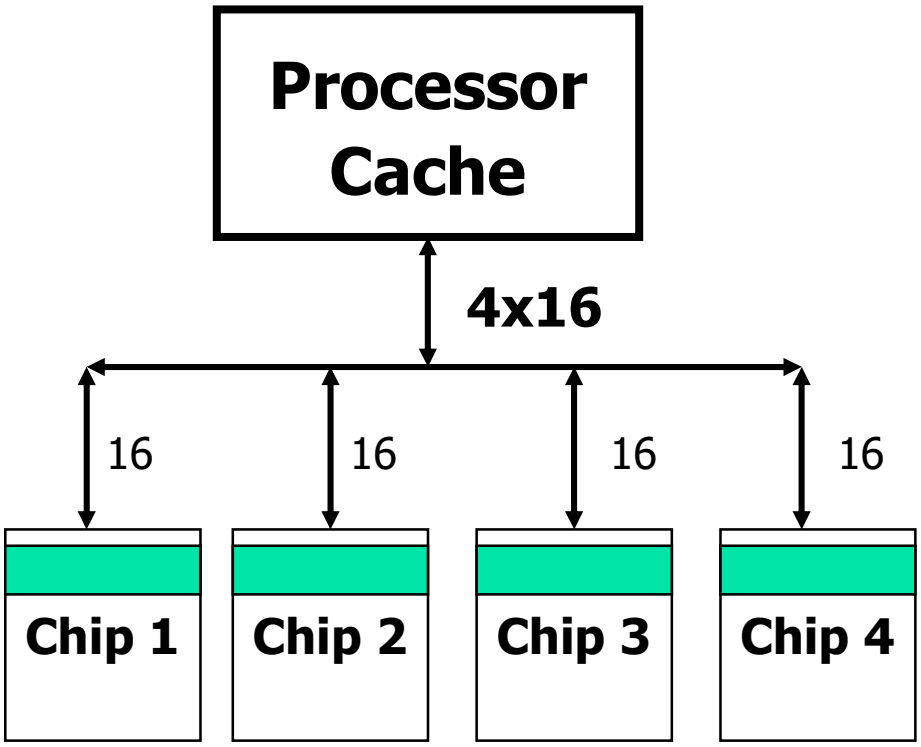
La commande ACTIVATE ouvre une ligne pour une opération de lecture ou d'écriture.

La commande PRECHARGE ferme la ligne.

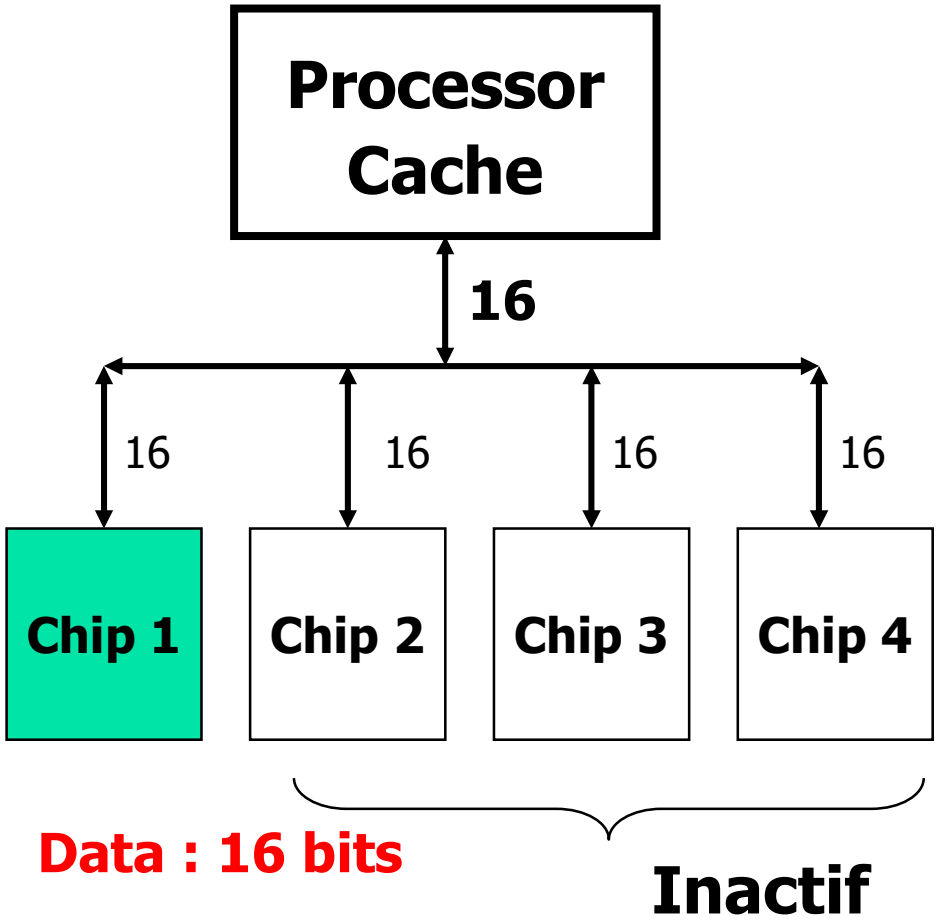
Typiquement le refresh intervient tous les 64 ms.



Consommation des DRAM : Mémoire multi_bancs

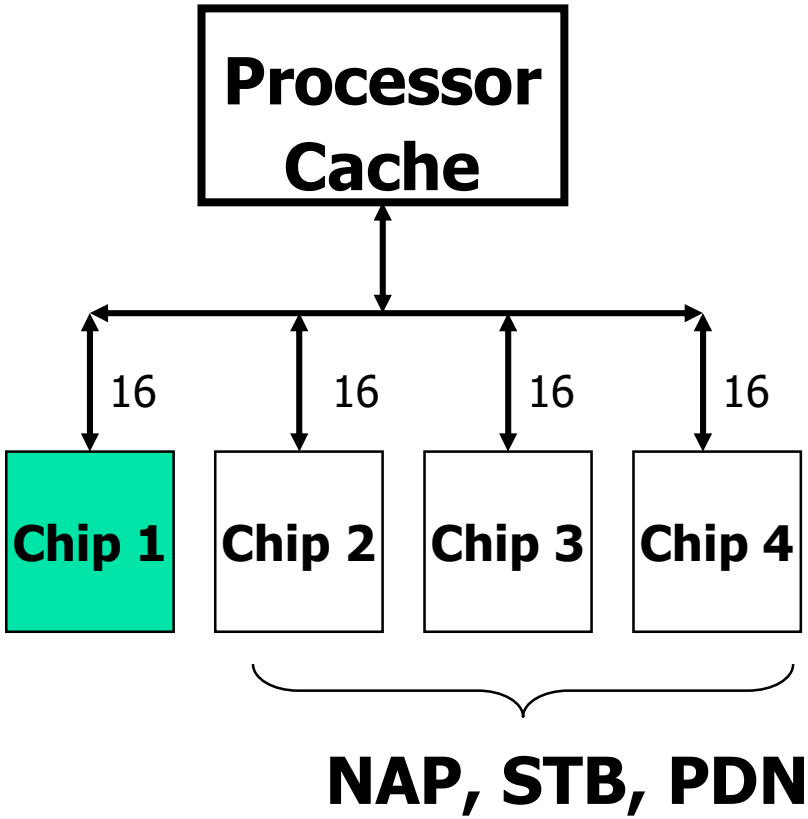


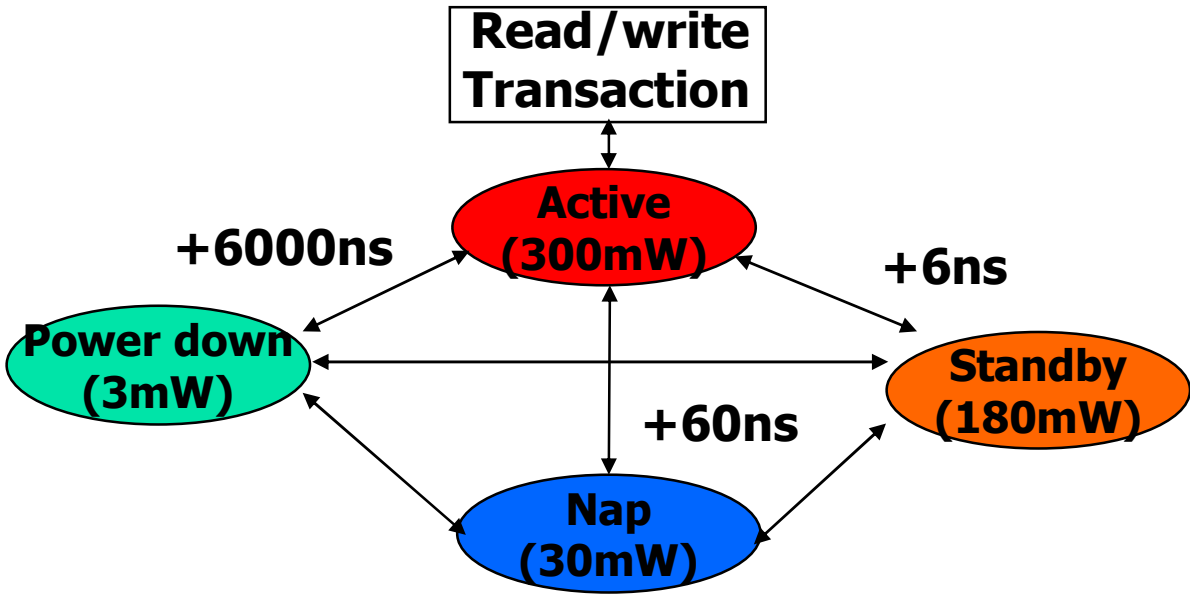
Data : 64 bits



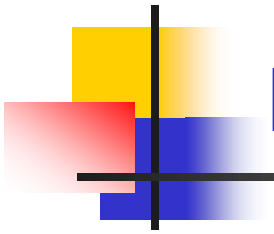
Data : 16 bits

RDRAM (Rambus DRAM) [Le00]





capacité : 64 Mbits réparti en 4 bancs



Mémoire multi-bancs

Considérons les programmes décrits pages 37/38.

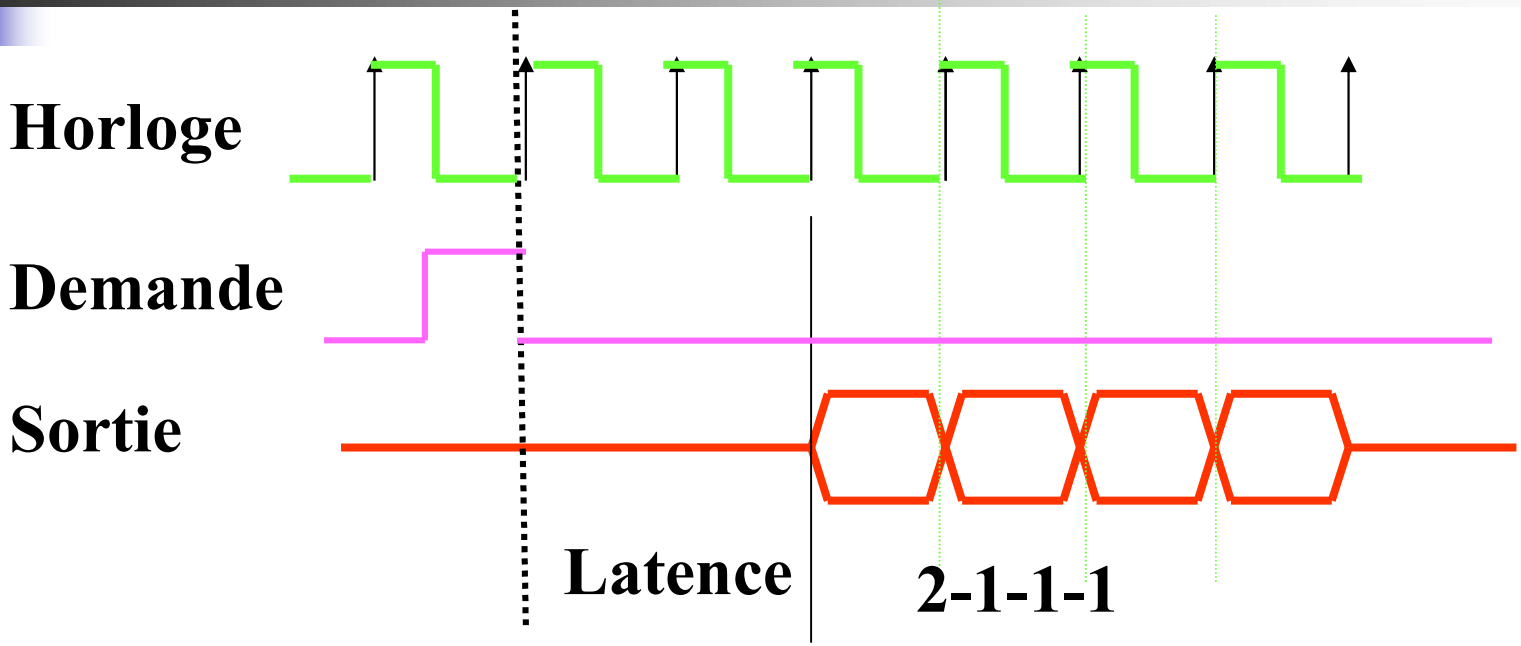
Déterminer le temps le temps d'exécution pour une mémoire externe.

Considérer les deux cas suivants :

- mémoire multi-bancs avec A,B,C dans le même banc
- A, B, C dans des bancs différents.

En cas de cache miss, l'accès à la mémoire externe nécessite 45 ns pour deux adresses appartenant au même banc ou 75 ns si les bancs différent.

Mode Burst (rafale des SDRAM)



Une demande : N réponses après K cycles de latence
Exemple : SDRAM DDR : 10ns

Mode Burst (rafale des SDRAM)

Notation k-x-y-z dans l'ordre :
CAS Latency : temps minimum pour accéder à une donnée (ligne, colonne)
RAS Precharge Time : temps minimum entre 2 accès ligne
RAS to CAS delay : temps pour accéder d'une ligne à une colonne
RAS Active time : temps minimum pour accéder à une ligne

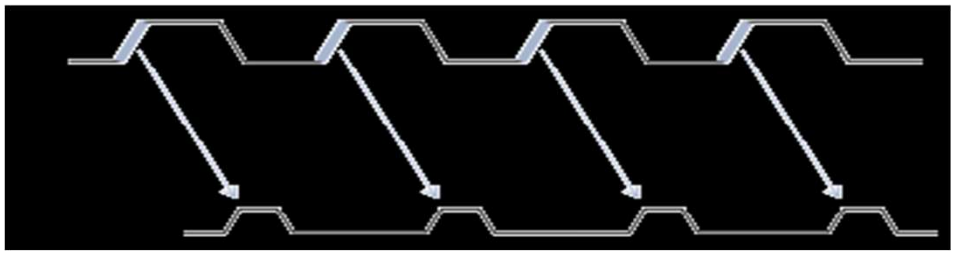
Que signifie : 3-2-2-2 ?



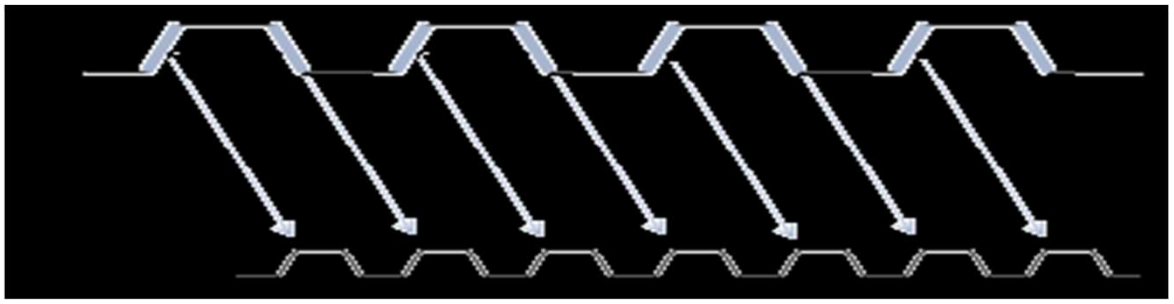
Déterminer la latence pour accéder à ces données.

DDRAM et SDRAM

- SDRAM : Synchronous DRAM - bus de communication à 66, 100 et 133 Mhz.



- DDR-SDRAM : Double Data Rate SDRAM. On double le taux de transfert en conservant la même fréquence de fonctionnement qu´une SDRAM (les données sont transmises vers le processeur sur les deux fronts d'horloge).



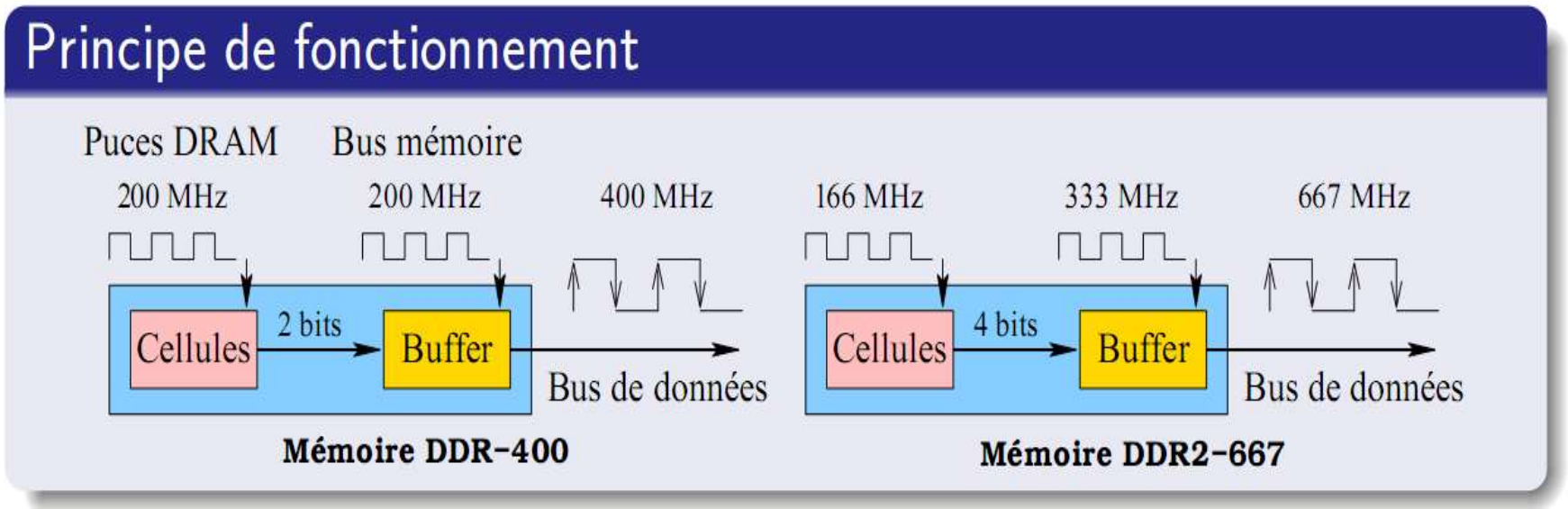
DDRAM et fréquence réelle (FR)

Basée sur la fréquence d'horloge réelle des puces DRAM (F_{DRAM})

DDR → FR = $2xF_{SDRAM}$;

DDR2 → FR = $4 \times F_{SDRAM}$;

DDR3 → FR = 8? × F_{DRAM} ;



Exercice

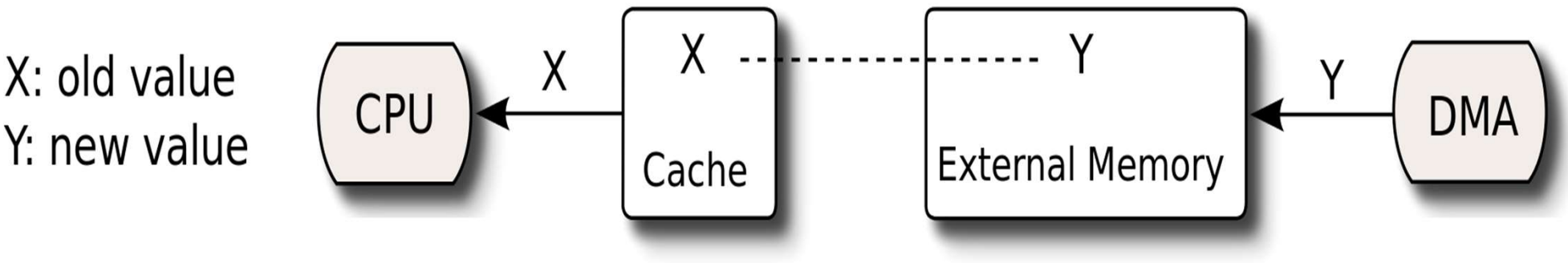
- DDR-400 → FR = ? MHz (F_{DRAM} = ? MHz) ;
- DDR2-1066 → FR = ? MHz (F_{DRAM} = ? MHz) ;
- DDR3-1333 → FR = ? MHz (F_{DRAM} = ? MHz)
- DDR4-2667Mhz ...

Calculer le débit maximum en Go/s de ces mémoires pour un bus de 32 bits puis de 64 bits.
Quelles sont les conditions pour obtenir ce débit ?
Considérer le cas des data et des instructions.

Remarque : système embarqué => LPDDR (mobile DDR)
Density : Up to 8 Gbit, Max Clock Speed : 333Mhz , Data width : 16 & 32 Bits
Comparer les débits.

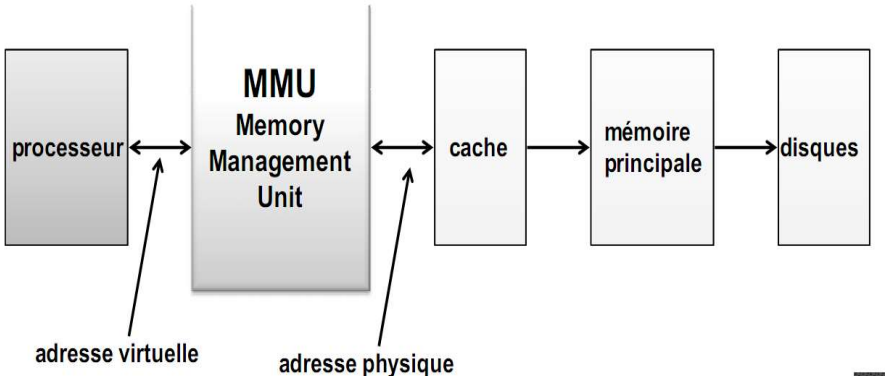
Notion de DMA

DMA (direct memory access) : Assure le transfert automatique de données vers/depuis la mémoire



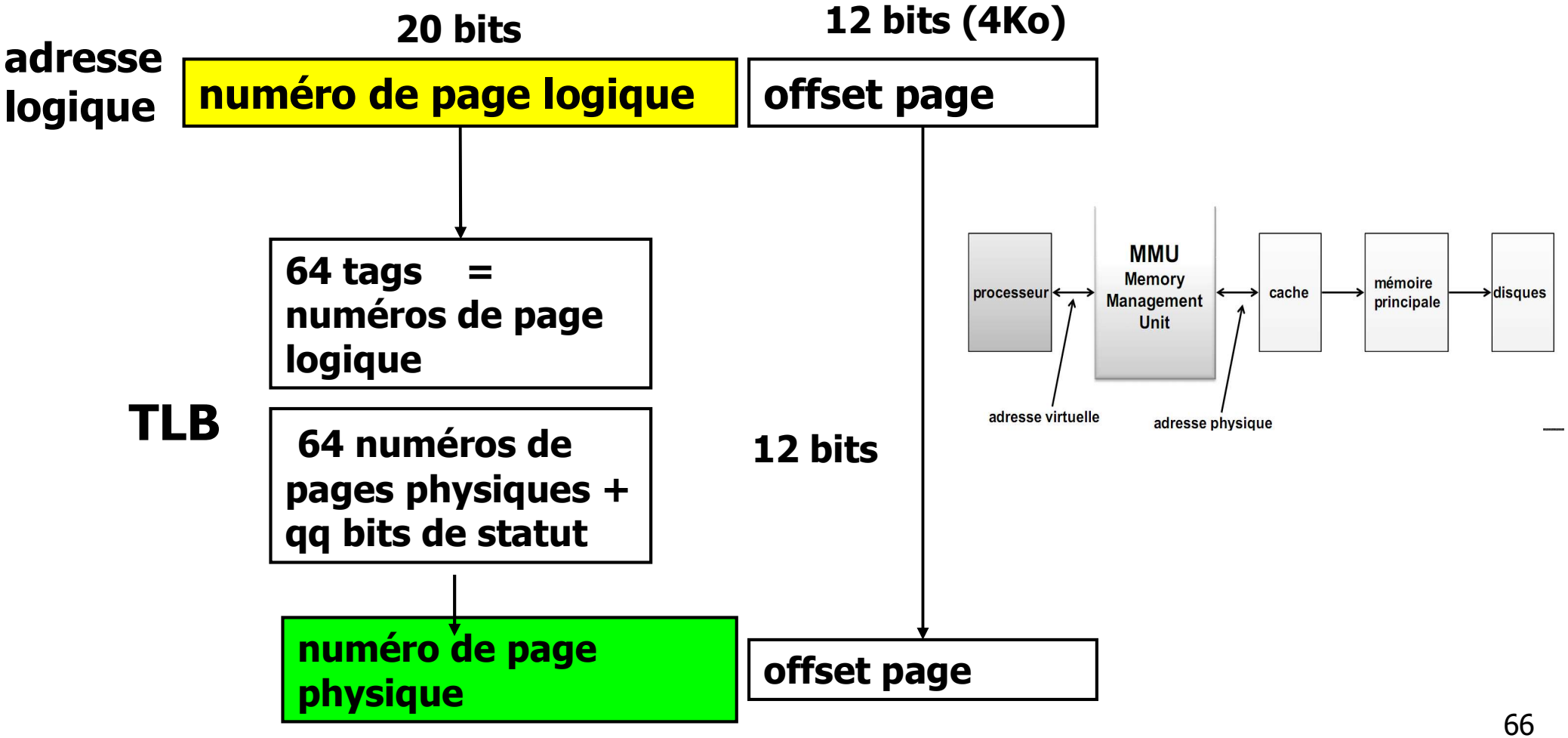
Organisation de la mémoire

- ✓ Processus/tache => espace mémoire privé.
- ✓ Caches + DRAM + mémoire de masse (disque dur ou flash) : cette dernière permet de stocker une partie des informations des processus lorsque la RAM est pleine => mécanisme de mémoire paginée, l'espace mémoire est découpé en blocs de taille fixe (**page mémoire, typiquement 4ko)**
- ✓ le MMU (Memory Management Unit) : établit les correspondances entre les adresses logiques utilisées par le programme dans son espace d'adressage privé et les adresses réelles des pages dans la RAM
- ✓ le MMU (Memory Management Unit) s'appuie sur un TLB (translation look-aside buffer)

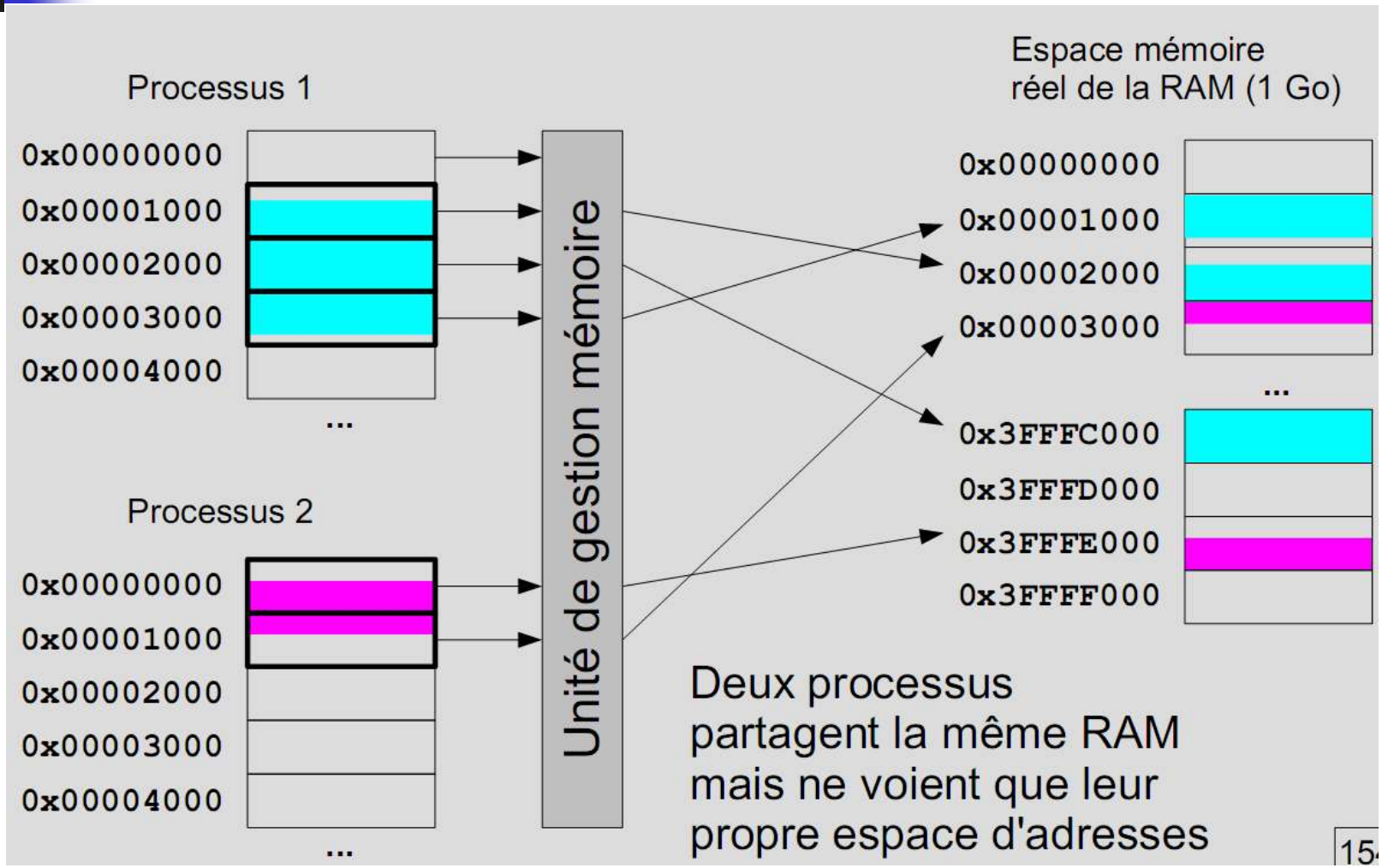


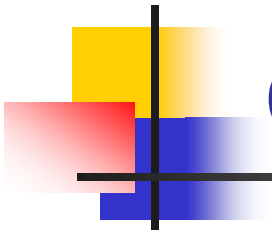
Organisation mémoire : MMU ET TLB

Exemple: adresses logiques 32 bits, pages de 4 Ko, TLB 64 entrées



Organisation mémoire





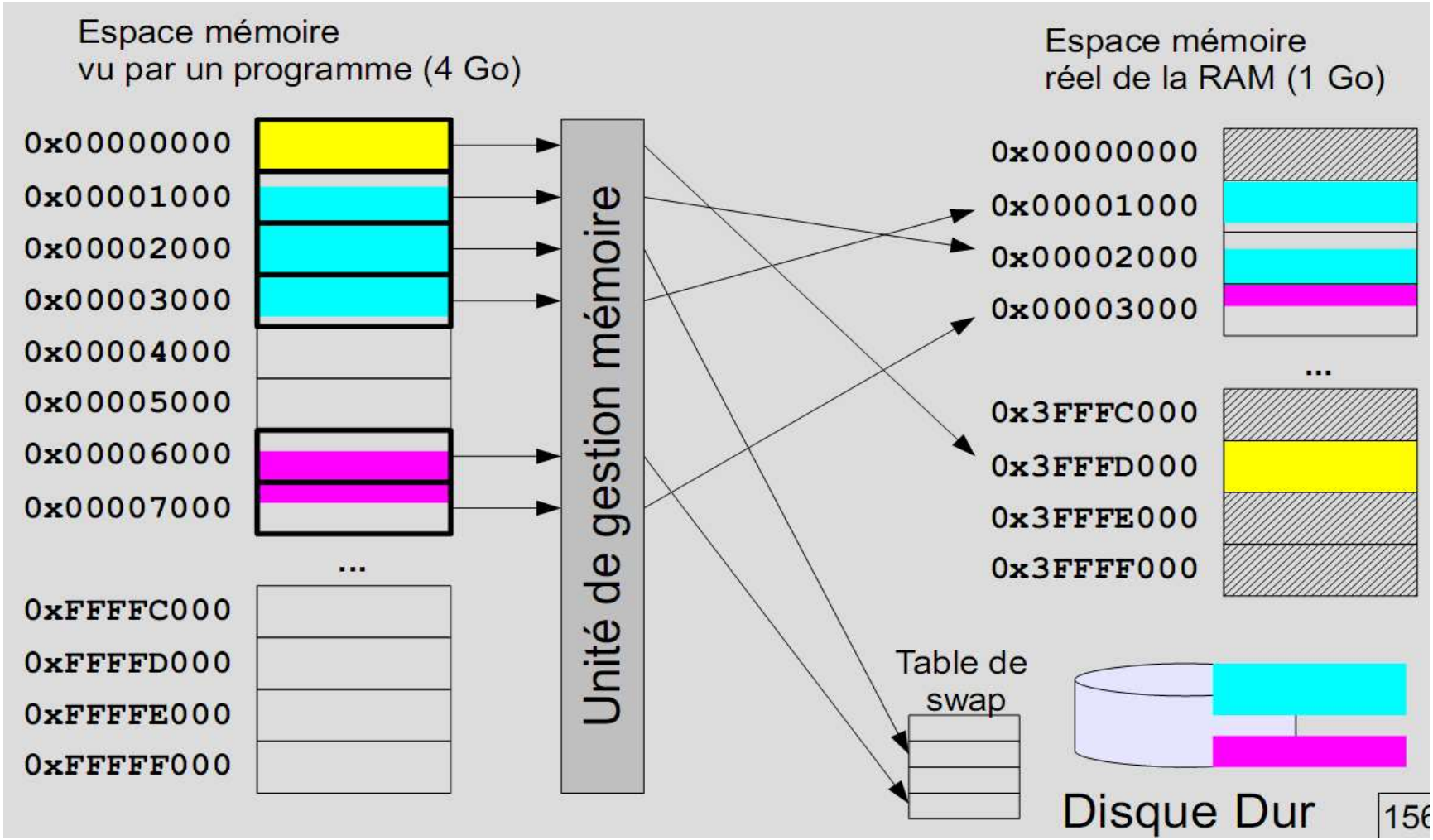
Organisation mémoire

- Lorsque la RAM est pleine on transfert des pages vers la mémoire de masse (DD ou flash)sur une zone de stockage temporaire : mécanisme de ***swap***

- Si un processus accède à une adresse qui correspond à une page qui n'est pas en RAM, la MMU déclenche une routine du noyau qui doit accéder à la mémoire de mass pour rapatrier la page en RAM

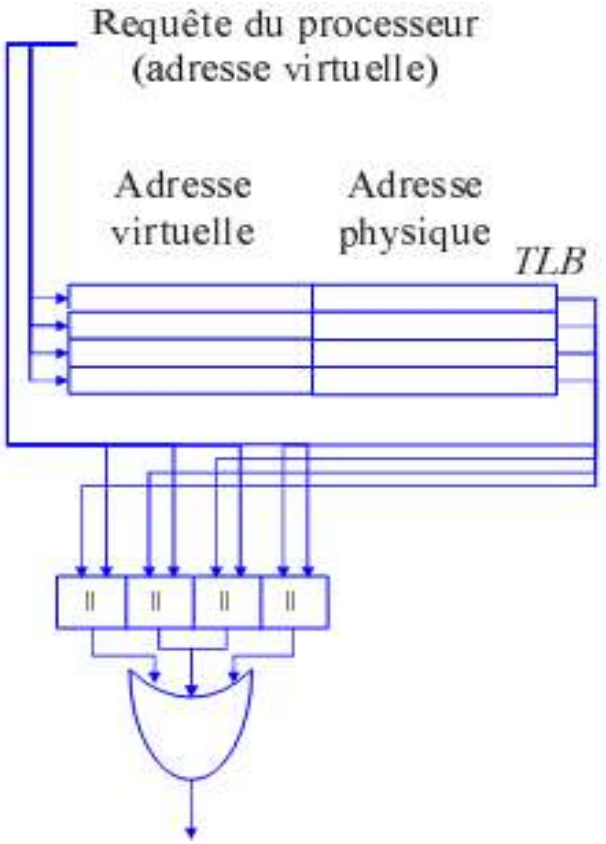
- Si il faut renoncer à maintenir en RAM une autre page (faute de place) on choisit la plus anciennement accédée pour swaper.

Organisation mémoire



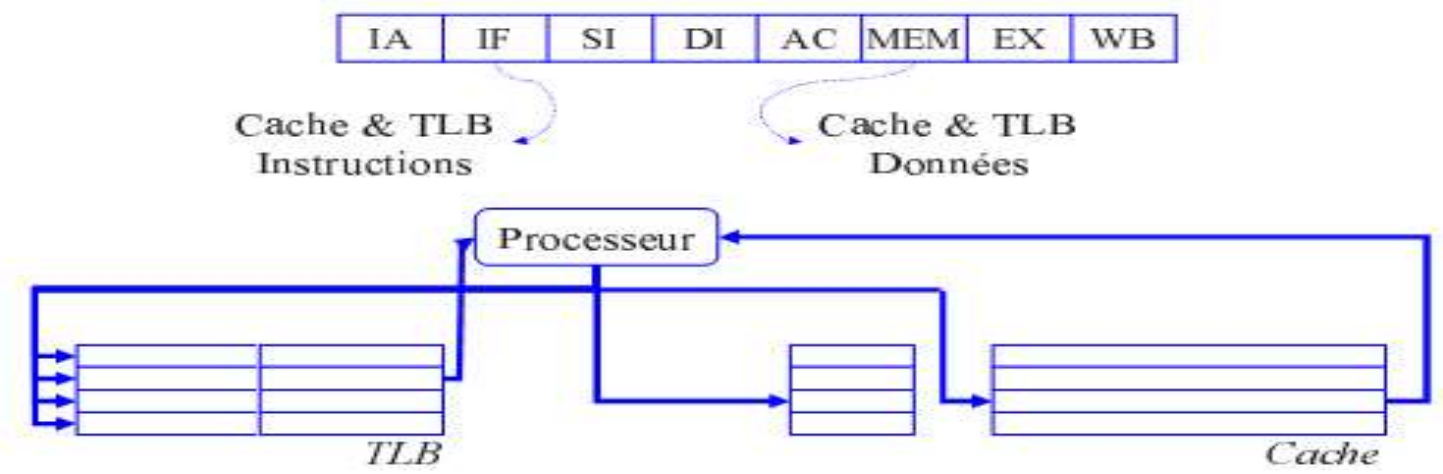
Mémoire physique/Mémoire virtuelle

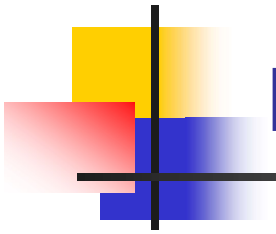
- Le processeur utilise des adresses virtuelles
- Les données ont une adresse en mémoire physique
  - Traduction adresse viruelle ⇔adresse physique
  - TLB (Translation Lookaside Buffer) : cache de traduction d'adresse
  - Une entrée du TLB correspond à une page
  - Souvent le TLB => associatif.



Gestion des adresses

- L'adresse stockée dans le cache est souvent l'adresse physique.
- Le processeur accède au cache avec une adresse virtuelle,
- Le processeur accède au TLB pour obtenir la traduction effectue la comparaison ensuite.
- Souvent on conçoit les caches afin d'effectuer en parallèle l'accès au cache et au TLB.





PLAN

1. Introduction & Paramètres importants
2. Architecture multi-unités de traitements :
    1. Architecture pipeline
    2. Architecture du système mémoire
    3. Architecture Superscalaire et VLIW
3. Architecture multicores

Superscalaire – VLIW - multiprocesseur

- **Processeur Superscalaire :**

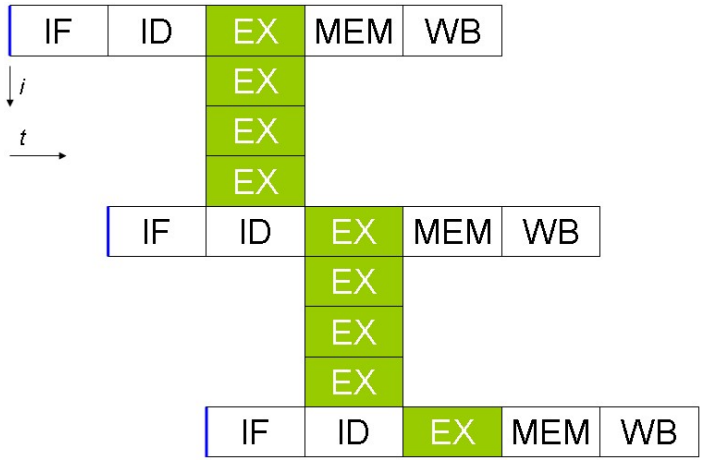
  Le matériel est responsable à l'exécution du lancement parallèle des instructions.

  => Contrôle des dépendances de données et des instructions par le matériel

- **Processeur VLIW :**

  Le compilateur doit présenter au matériel des instructions exécutables en parallèle.

  => Contrôle des dépendances de données et des instructions par le compilateur

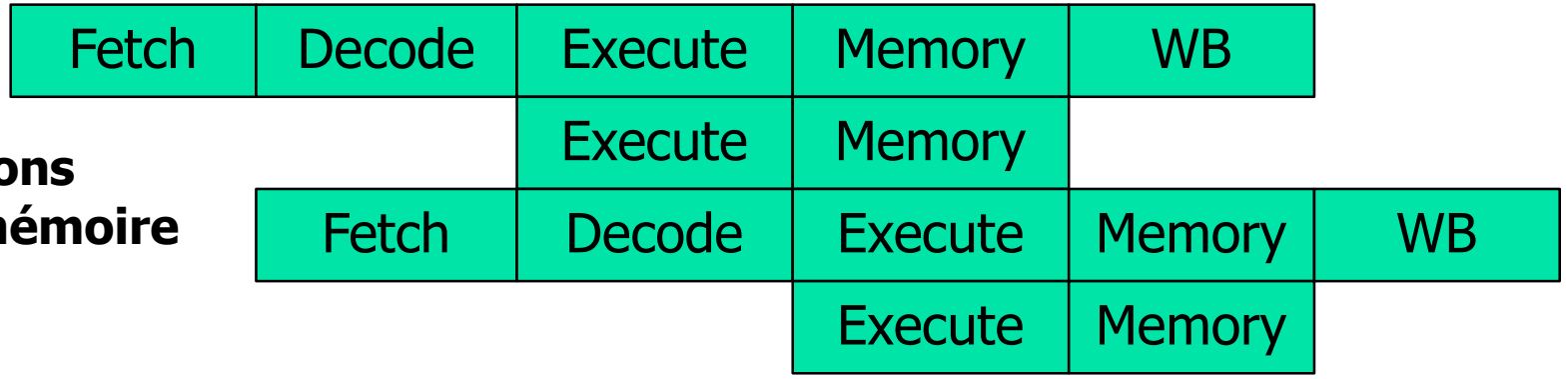


- ***Multicores :***

  C'est l'OS qui distribue les instructions et/ou les applications (tâches) à exécuter en parallèle.

Architecture superscalaire : principe

- Architecture pipeline : une instruction par cycle.
- Superscalaire de degré n : n instructions terminées par cycle



2 opérations
2 accès mémoire

- Pour réaliser un processeur superscalaire, il faut :
  - Disposer de plusieurs unités de calcul
  - Assurer un flux d'instruction suffisant,
  - Déterminer quelles instructions peuvent être exécutées en parallèle,
  - Vérifier la cohérence des données entre les instructions

Programmation du parallélisme ?

Détection de chute : Détection de contour & transformation en image Noir et blanc :

$$O_i(x,y) = \begin{cases} 1 & \text{if } | I_i(x,y) - B_i(x,y) | \geq \tau \\ 0 & \text{if } | I_i(x,y) - B_i(x,y) | < \tau \end{cases}$$

Puis mise à jour de l'image de fond :

$B_{i+1}(x,y) = (1-a) * B_i(x,y) + a* 1/3 (I_{i-2}(x,y) + I_{i-1}(x,y) + I_i(x,y))$



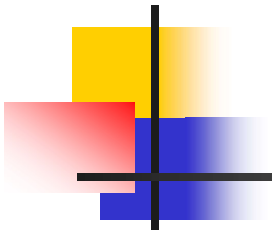




- Identification des types d'instructions exécutables en parallèles
- Politique d'exécution : Exécution dans l'ordre non ordonné puis ordonné

Attention : - Dépendances de données
- Branchements

(ILP : Instruction level parallelism)



Exécution Superscalaire

- Exécution dans l'ordre
  - Acquisition par groupe d'instructions
  - Traitement du groupe suivant quand toutes les instructions d'un groupe ont été lancées et démarrées.

- Exécution non ordonnée
  - Acquisition des instructions par groupe, rangées dans un tampon
  - Exécution non ordonnée des instructions, en fonction du flot de données
  - Fenêtre d'instructions liée à la taille du tampon.

Exemple d'exécution ordonnée

- Considérons un cœur de processeur doté de : 2 UALs, 2 FPU, et capable de réaliser 2 accès mémoires

Instr1, …. Instr n

UAL1	UAL2	FPU1	FPU2	MEM 1	MEM2
Instr 1	Instr2	Instr3	Instr 4	Instr 5	Instr6
Instr 7				Instr 8	
		Instr 9	Instr 10	Instr 11	
Instr 12					…

- Extraction de 6 instructions
  - Ces 6 instructions se répartissent dans :

    UAL1, UAL2, FPU1 et FPU2, MEM1, MEM2
  - Les 6 instructions suivantes sont traitées quand les instructions précédentes ont démarré.

Chargement des instructions

Nombre d'instructions chargées à chaque cycle => 4

Lignes de cache

0	1	2?	3?
2	3 ?	4?	5?
3	4 ?		

Instruction 4 => branchement

4	X	X	X

Utilisation possible de la prédiction de branchement pour éviter les bulles dans le pipeline.
Tampon des instructions

4	10	11	12

Destination

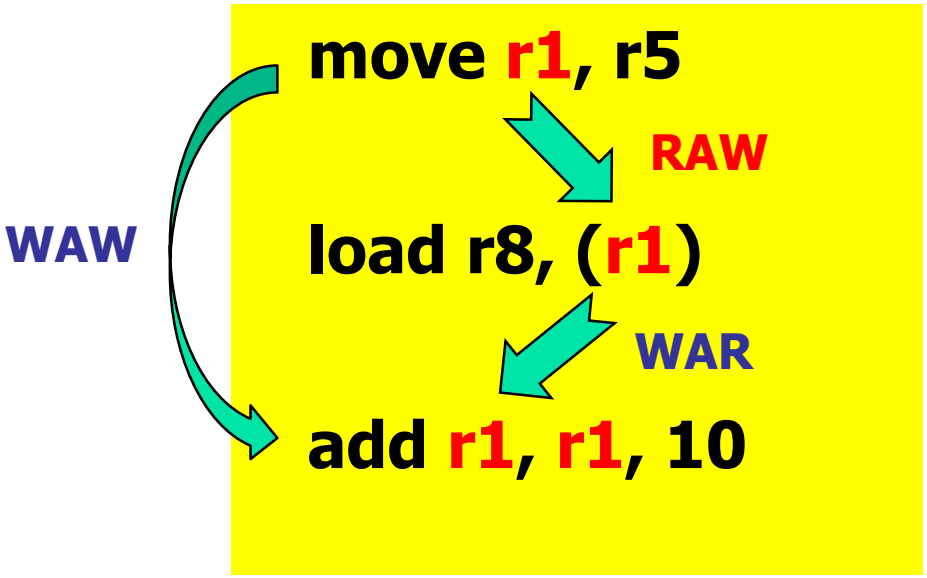
```
Etiq1 :
        move r7,r3        0
        load r8,(r3)      1
        add r3, r3,4      2
        load r9,(r3)      3
        ble r8,r9,Etiq2   4

        move r3,r7        5
        store (r9),r3     6
        add r3,r3,4       7
        store (r8),r3     8
        add r5,r5,1       9
Etiq2 :
        add, r6,r6,1      10
        add r7,r7,4       11
        blt r6,r4,Etiq1   12
```

Décodage, dépendance et mises en attente

Les instructions sont sorties du tampon d'instructions.

- On détermine les dépendances de données entre les instructions.
- On détermine les fausses dépendances liées aux aléas des registres.



RAW (Read After Write) : vraie dépendance
WAW (Write After Write) : risque d'écriture dans le désordre (fausse dépendance)
WAR (Write After Read) : risque d'écriture avant lecture (fausse dépendance)

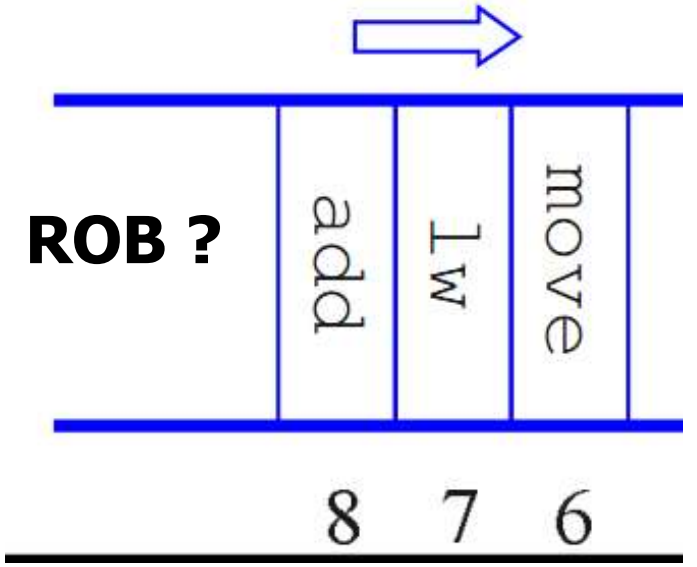
Solution : utilisation d'un ROB (ReOrder Buffer)

Aléas de registres : renommage

- Mémoire interne au processeur :
  - registres logiques (ceux définis par le jeu d'instructions)
  - banc de registres (registres physiques) et tampon de réordonnancement (ROB : ReOrder Buffer).

- le ROB contient les instructions et les valeurs produites par les instructions.

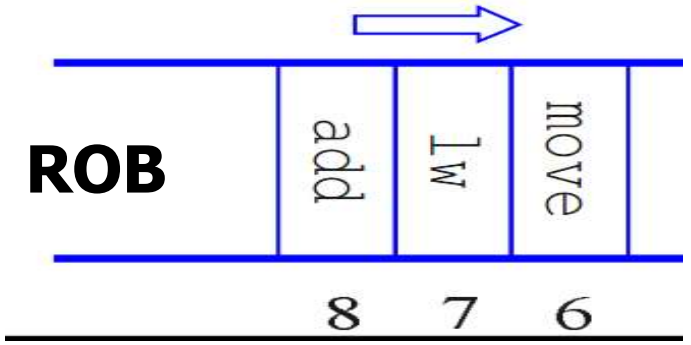
```
L2:
    move    r3, r7
    lw      r8, (r3)
    add     r3, r3, 4
```



Aléas de registres : renommage

- Le registre logique de destination est remplacé par le nom d'une entrée dans le ROB : on renomme le registre.
- Une table indique où se trouve la valeur courante d'un registre logique (dans le banc de registres ou dans le ROB); lors du chargement des instructions, ses opérandes peuvent être dans le ROB ou dans le banc de registres.

```
L2:
        move    r3,  r7
        lw      r8,  (r3)
        add     r3,  r3,  4
```



Registre logique	Zone de stockage physique
R_3	ROB_6 puis ROB_8
…	…
R_8	ROB_7

Registre physique	valeur
R_3	Produit par move puis add
…	
R_8	Produit par LW

Exercice

```
Etiq :  move R3, R7
        load R8,(R3)
        add R3, R1, 4
        load R9, (R3)
        add  R7, R1, R2
        Sub  R6, R1, 4
        ble R8,R9, Etiq
        …
```

RAW (Read After Write) :
vraie dépendance
WAW (Write After Write) :
risque d'écriture dans le désordre (fausse dépendance)
WAR (Write After Read) :
risque d'écriture avant lecture (fausse dépendance)

Déterminer pour le programme précédent les dépendances (RAW, WAW, WAR).

Montrer avec l'utilisation du ROB comment on peut ôter les fausses dépendances de ce programme.

Programmation du parallélisme : exercice

Détection de chute : Détection de contour & transformation en image Noir et blanc:

$$O_i(x,y) = \begin{cases} 1 & \text{if } |I_i(x,y) - B_i(x,y)| \geq \tau \\ 0 & \text{if } |I_i(x,y) - B_i(x,y)| < \tau \end{cases}$$

Puis mise à jour de l'image de fond :

$B_{i+1}(x,y) = (1-\alpha)\, B_i(x,y) + \alpha * 1/3 (I_{i-2}(x,y) + I_{i-1}(x,y) + I_i(x,y))$

Ou

$$B = \frac{1}{15}\sum_{i=1}^{15} I_i$$



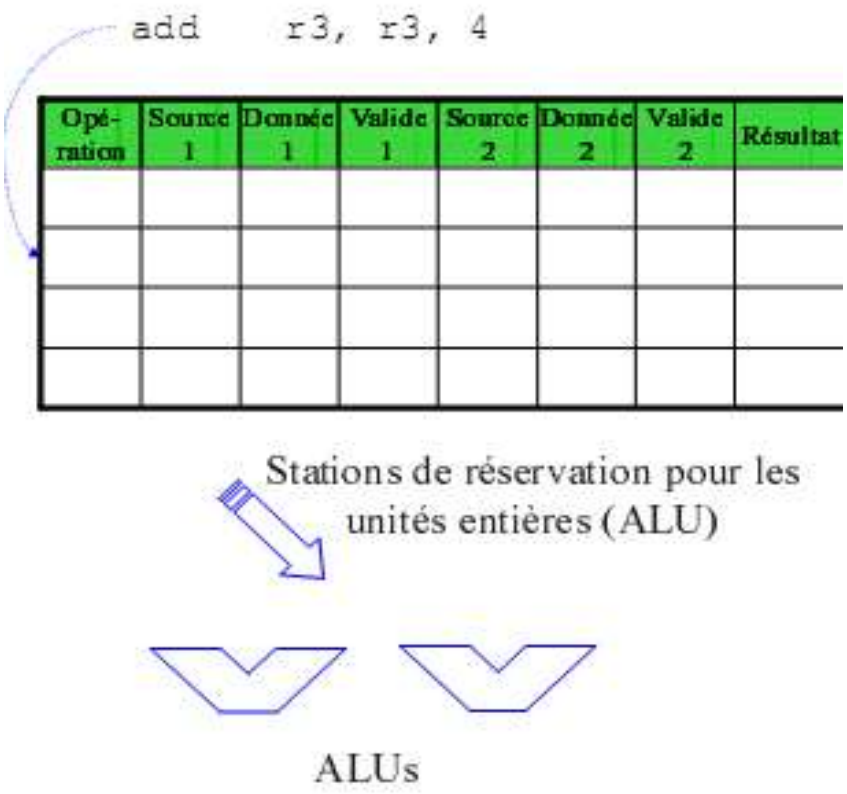
Considérons une architecture comportant 2 unités entières, 2 unités flottantes et capable de faire 2 accès mémoire

=> Déterminer le parallélisme pouvant être extrait pour cette application

Mise en attente (Dispatch)

- Après résolution des dépendances, envoi des instructions pour exécution (dispatch),
- Stations de réservation pour chaque unité fonctionnelle (ou groupe d'unité fonctionnelle), une file d'attente des instructions,
- Une instruction est exécutée si :
  - Toutes ces opérandes sont disponibles,
  - Une unité fonctionnelle est disponible.

(Algorithme de Tomasulo)



Accès mémoire et Exécution dans le désordre

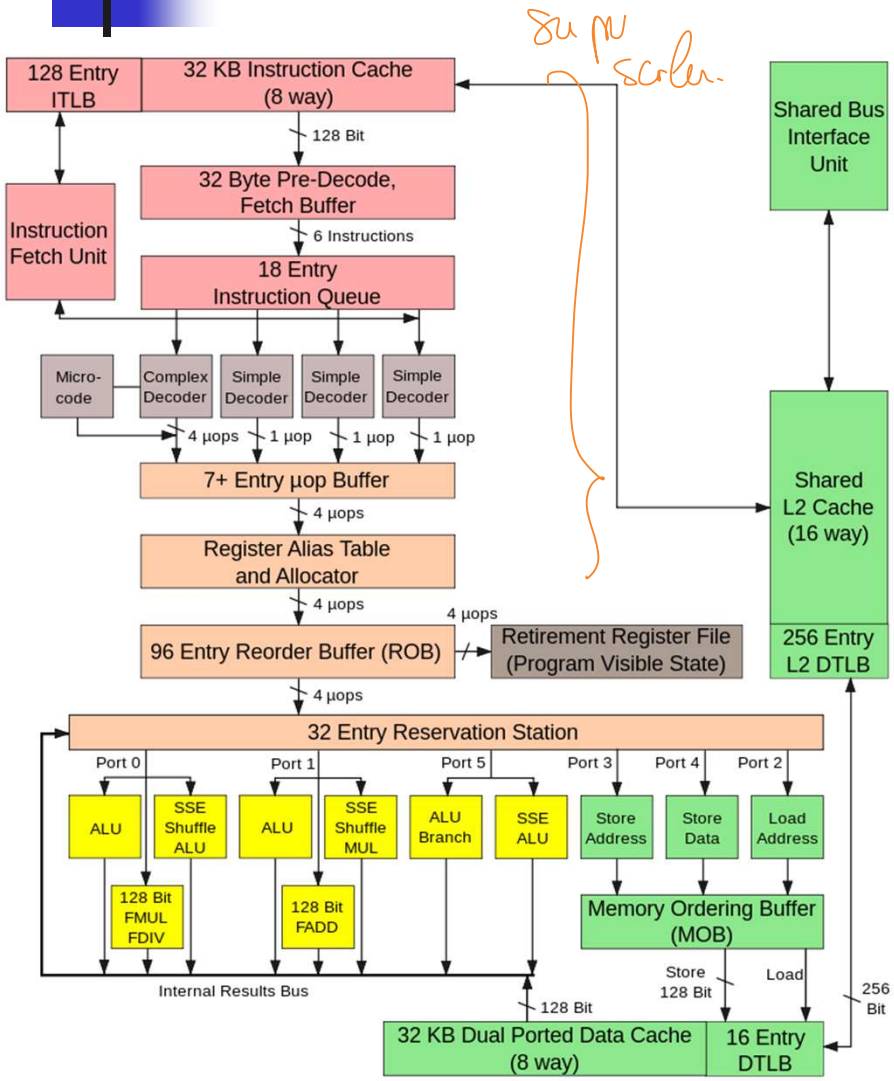
L'exécution des instructions dans le désordre permet de masquer partiellement la latence en cas de défaut de cache.

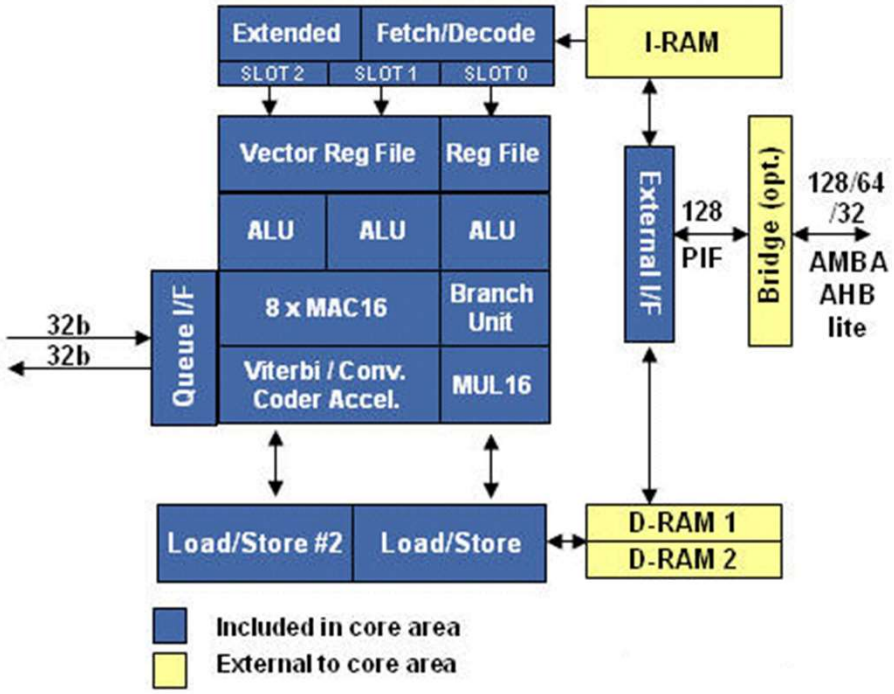
```
lw      r1, 12(r2)
add     r2, r1, 3
...
lw      r3, 20(r5)
...
add     r4, r1, r3
```

Cycle	Action
10	lw r1 : *miss*
11	add r2, r1, r3 : bloqué
12	...
13	lw r3 : *hit*
14	...
15	lw r1 : terminé
16	add r2, r1, r3 : terminé
17	add r4, r1, r3 : terminé
	aucun délai dû au miss

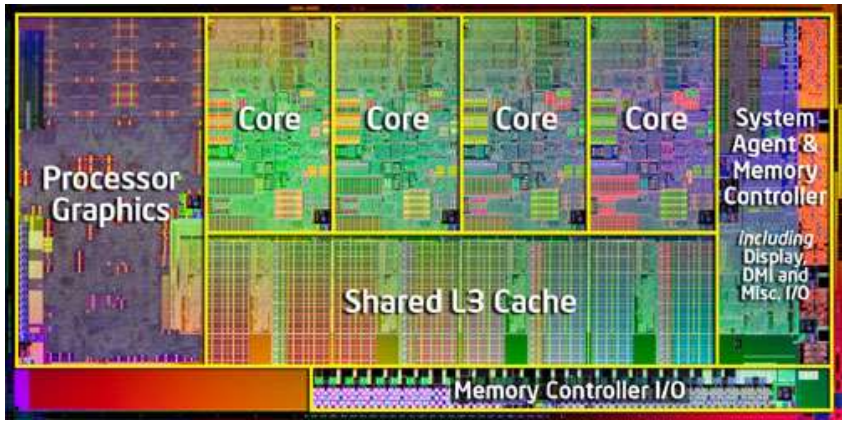
Latence mémoire: 5 cycles

Exemple de core de processeur

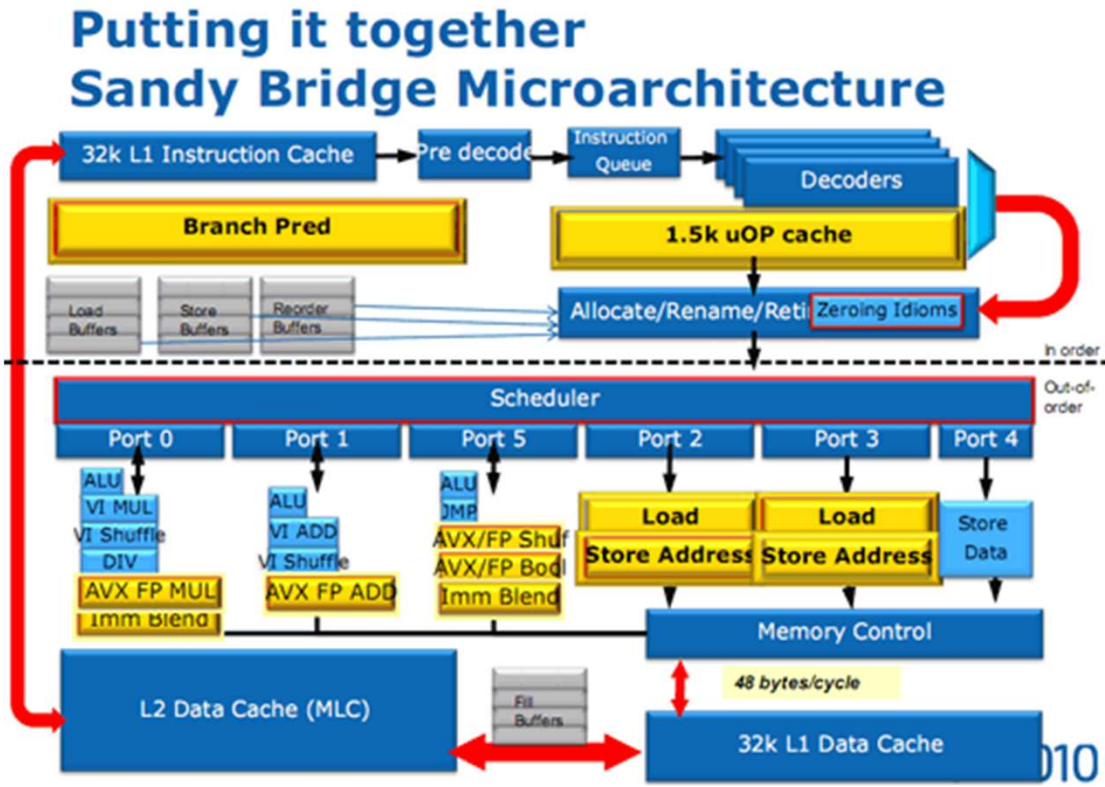




Exemple de core



Processeur Intel i5, i7



Exercice

Considérons un filtre de traitement d'image :

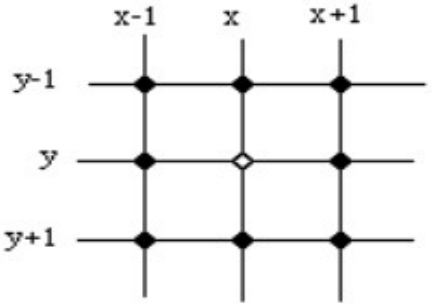
Pour tous les pixels d'une image I, cette opération consiste à remplacer la couleur de chaque pixel (x,y) par une combinaison linéaire des couleurs de tous les points immédiatement voisins (y compris lui-même) :

$g(x,y)=\Sigma i \Sigma j\ h(i,j)*I(x+i, y+j)$ avec i et j variant de -n à +n

$$\left(\sum_{i=-n}^{i=n}\sum_{j=-n}^{j=n} h(i,j).I(x+i,y+j)\right)$$

où h est un masque qui peut adoucir ou durcir l'image.

A titre d'exemple, si n=1, les pixels considérés sont positionnées comme ci-après avec un exemple de masque h.



$$h=1/10*\begin{vmatrix}1 & 1 & 1\\ 1 & 2 & 1\\ 1 & 1 & 1\end{vmatrix}$$

Sur les architectures précédentes :

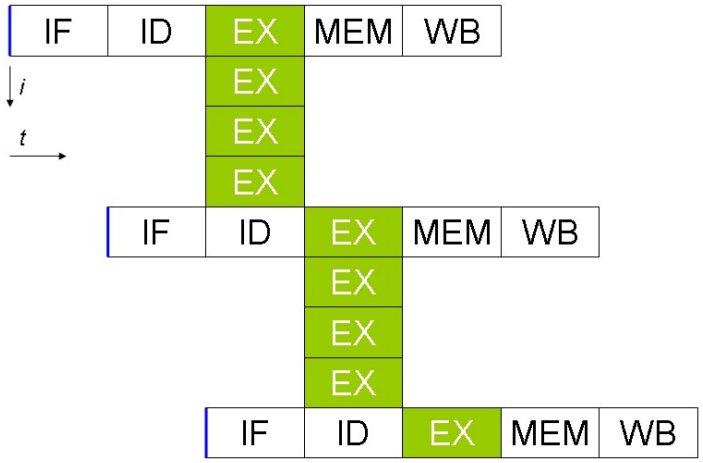
- Relever le parallélisme intracore, puis estimer le temps de réponse pour n=1

Architecture VLIW (Very Long Instruction Word)

- Le compilateur doit présenter au matériel des instructions exécutables en parallèle.

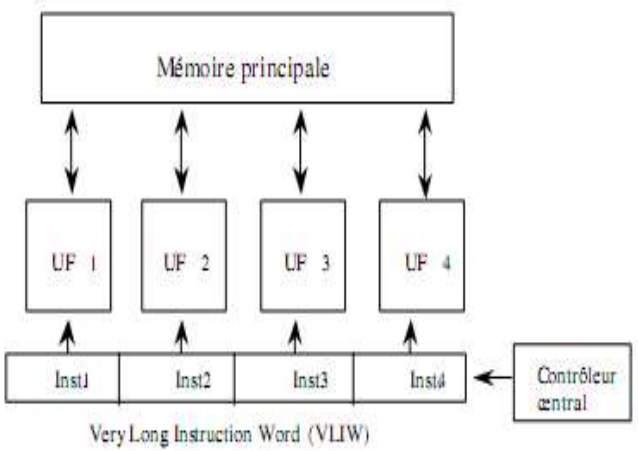
  => Création de macro-instructions en associant des instructions élémentaires à exécuter simultanément.
- Famille de processeur dont la taille du mot instruction est couramment supérieur à 128 bits => contient les instructions des opérations à effectuer par les différentes unités de calcul disponibles dans l'unité centrale.
- Rôle du compilateur :

  générer un code qui prend en compte la disponibilité des ressources et ne provoque pas de problèmes matériels lors de son exécution.



VLIW : principe

- Ordonnancement statique (compilateur)
  - Aléas de données
  - Aléas de contrôle
- Exécution pipeline des instructions
- Exécution parallèle selon N unités fonctionnelles
- Tâches du compilateur
  - Graphe de dépendances locales : Bloc de base
  - Graphe de dépendance du programme.
- Ordonnancement de traces
  - Avantage : Matériel simplifié (contrôle) => système embarqué
  - désavantage : La compatibilité binaire entre générations successives est difficile



Exemple machine VLIW/programme

- une machine VLIW dispose généralement :
  - d'une mémoire permettant au moins deux opérations simultanées,
  - un additionneur/soustracteur
  - un multiplieur
  - un diviseur …

Les instructions peuvent comporter 6 champs

- 2 pour les opérations de chargement/rangement en mémoire
- 4 pour les opérations arithmétiques …

Exemple de programme :

A = J + K; B = L × M; C = N – B; D = A / B

5 instructions peuvent ainsi s'exécuter chacune en 1 cycle d'horloge.
=> Ordre d'exécution des opérations ?

Exemple machine VLIW/programme

Exemple de programme :

A = J + K; B = L × M; C = N – B; D = A / B

Dépendance de type RAW (*Read After Write*) sur A et B

	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5
Accès MEM	Load Rj,J	Load Rl,L	Store Ra,A	Store Rb,B	Store Rc,C
Accès MEM	Load Rk,K	Load Rm,M	Load Rn,N		Store Rd,D
Add sub		Ra=Rj+Rk		Rc=Rn-Rb	
Mul			Rb=Rl*Rm		
Div				Rd=Ra/Rb	

Amélioration ?

Exemple : parallélisme ?

```
#define Nmax XX
int foo(int *a, int *b, int *c)
{
    int i;
    for (i=1; i<Nmax; i++)
    {
        a[i] = (b[i]*c[i])+a[i-1];
    }
    return a[Nmax-1];
}
```

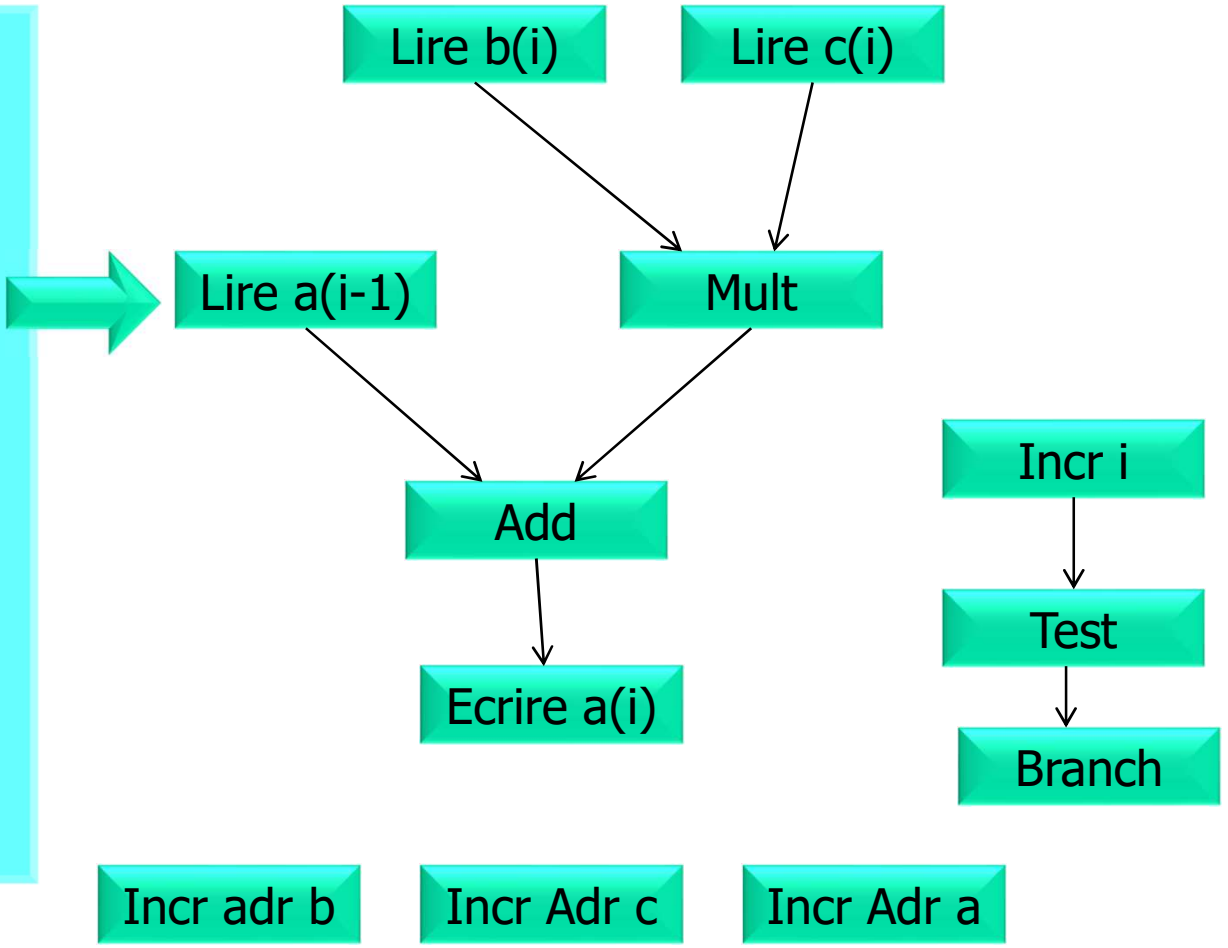
Optimisation compilateur

R1 : adresse de **b**, R2 : adresse de **c**,
R3 : adresse de **a**, R4: i

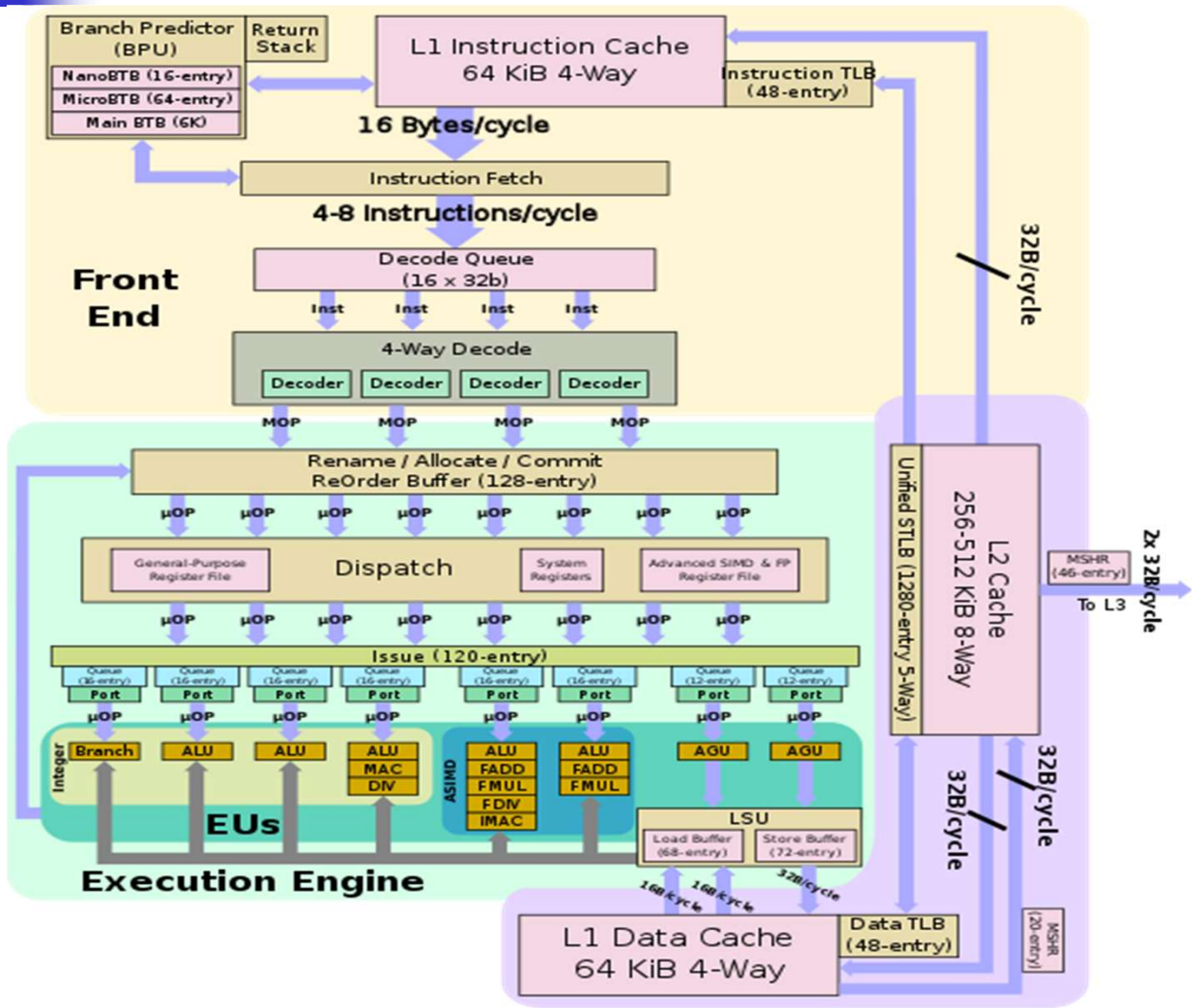
```
doloop:
LOAD R7, [R1]            => b[i]
ADD R1, R1, 4
LOAD R8, [R2]            => c[i]
ADD R2, R2, 4
LOAD R9, [R3]            =>a[i-1]
ADD R3, R3, 4
MUL R10, R7, R8          =>b[i]*c[i]
ADD R11, R10, R9   =>(b[i]*c[i])+a[i-1]
STORE [R3], R11           => a[i] = ...
SUB R4, R4,1      => compteur de boucle
JUMPZ R4,doloop
...
```

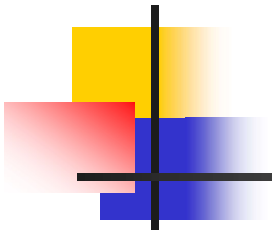
Exercice – Application - représentation

```
#define Nmax XX
int filtre(int *a, int *b, int *c)
{
    int i;
    for (i=1; i<Nmax; i++)
    {
        a[i] = (b[i]*c[i])+a[i-1];
    }
    return a[Nmax-1];
}
```



Exemple Architecture équivalent ARM 76





Exercice – Application filtrage vidéo

1 - Dessiner le DFG (data flow graphe) :

- Déterminer les dépendances pour l'application de filtrage vidéo
- Quel est le temps d'exécution optimal ?

2 - Calculer le temps d'exécution pour une architecture RISC (1 instr/cycle).

3 - Proposer un ordonnancement des instructions pour une architecture VLIW :

arch1 : 2 accès mémoire, 2 add, mut, div

arch2 : 4 accès mémoire, 3 add, mul, div …

Qu'en déduisez vous ?

4 - Pour améliorer la performance du code sur l'architecture VLIW, on propose de faire un dépliage de boucle.

Quels sont les gains ?

Achitectures multiflots

- ⇒ Déjà étudié : recherche du parallélisme intra task
- ⇒ Amélioration : exécution de différents flots, parallélisme inter task

Exemple de programme :

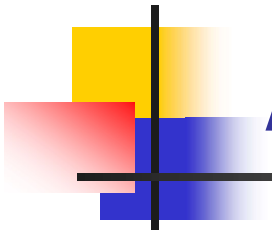
Pgm 1 : A = J + K; B = L × M;

Pgm 2 : C = N – E; D = F/G

Exécution ?

	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle …
Accès MEM					
Accès MEM					
Add sub					
Mul					
Div					

Reprendre l'application de filtrage vidéo et proposer une amélioration.



Achitectures offrant du parallélisme

- Superscalaire
- VLIW : Exploitation du parallélisme par le compilateur
- Multiflots/multithreading => Système d'exploitation

- Système d'exploitation :
  - ordonnancement sur un core
  - ordonnancement multicore

Notion d'ordonanncement

Téléphone mobile : lecture vidéo
⇒ **Audio : 96 kbps**
⇒ **vidéo : 30 fps (QVGA)**



Vidéo : Pv : 1/30 s, Cv : 20 ms
Audio : Pa : 1/44100 s, Ca : 7 µs
Accrochage réseau :
Pr : 1/21 s, Cr : 577 µs

Système temps réel : Application => ensemble de tâches ti caractérisées par :

C_i : Worst Case Execution Time (WCET), P_i : Period of the task i,

A_i : Actual (real) execution time of the task i,

Charge de travail/processeur : $U_i = C_i/P_i$

Exemple : Traitement vidéo : 10 ms pour décoder une frame,

Période pour 25 fps : 40ms => charge core : 10/40 = 25%

Problème : Ordonnancer (définition des dates d'exécution) afin de satisfaire les contraintes temporelles *+ minimiser la consommation.*

Technique Hors ligne (Statique)

Technique En ligne (dynamique)

Contrainte temporelle
Hard, soft, best effort

Ordonnancement EDF hors ligne

Principe de la politique (Earliest Deadline first) [Gr02] :

Priorité à la tâche dont l'échéance est la plus proche.

Pour N tâches :

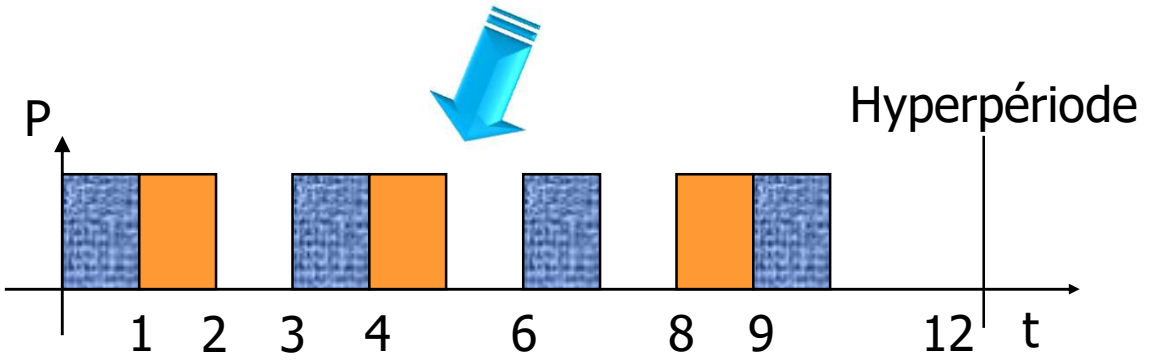
$$\text{Test d'ordonnancement} : U = \sum_{i=1}^{N} \frac{Ci}{Pi} \leq 1$$

Unité : 10 ms

Tâche	WCET	Période
T1	1	3
T2	1	4

Charge liée à T1 : 1/3
Charge processeur liée à T2 : 1/4

$$U = \frac{1}{3} + \frac{1}{4} = \frac{7}{12} < 1$$



Exercice : tâches indépendantes

Considérons un dispositif électronique qui a pour rôle d'évaluer l'activité d'une personne par exemple à mobilité réduite. Cette application présente les tâches suivantes :

- Evaluation de la mobilité : lecture des capteurs de type accéléromètre (T1) + détection de l'activité (T2) comme marche lente, normale, rapide, montée/descente escalier, nombre de pas ….
- Evaluation du temps passé outdoor (basé sur le GPS) (T3) + distance parcourue (évaluation de l'éloignement par rapport au domicile)
- Mesure du rythme cardiaque et détection d'alerte (T4)

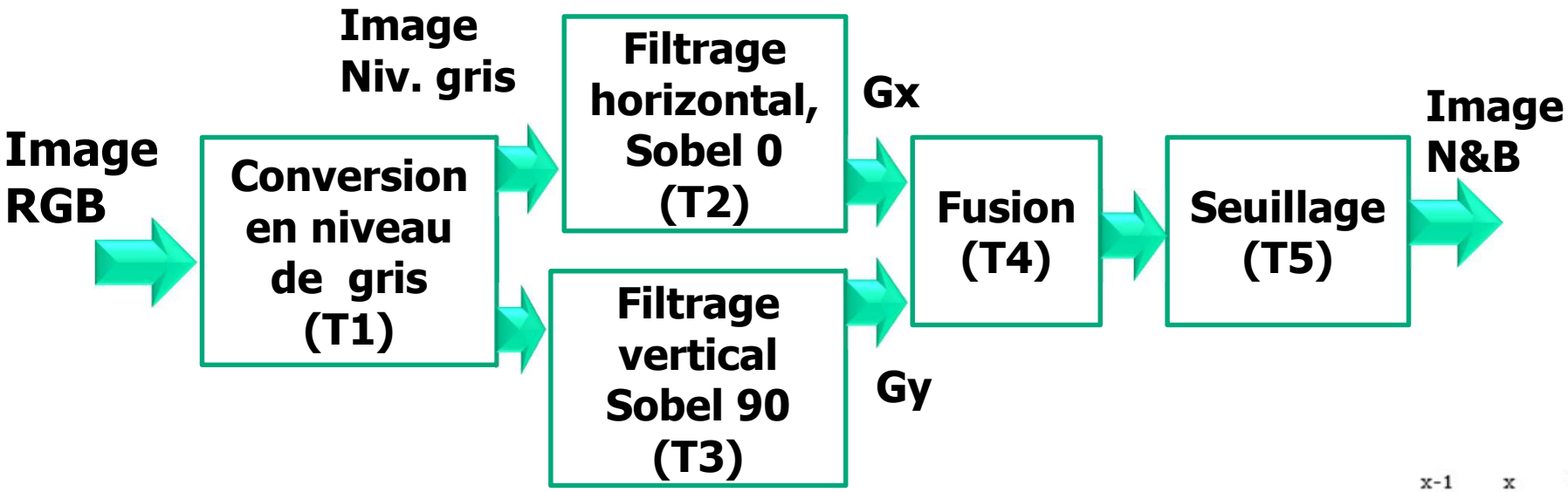
Est-il possible d'ordonnancer ces tâches sur un core type montre ?

Dans le cas positif, construire un ordonnancement.

	T1	T2	T3	T4
Temps	1ms	40 ms	10 ms	2 ms
Période	10ms	1000ms	1000ms	50 ms
Fréquence mesure	100/s	1/s	1/s	20/s

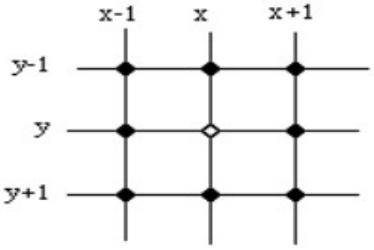
Exercice : tâches dépendantes

Considérons une application de traitement vidéo (détection de contour) défini par les tâches suivantes et les caractéristiques associées (fps=30):

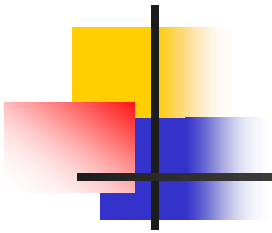


Conversion Niveau de gris : $I_{NivG}(x,y)= (R(x,y)+G(x,y)+V(x,y))/3$.
Filtrage : Pour le filtrage horizontal, h=(-1, 0, 1, -2, 0, 2, -1,0,1) et vertical h=(-1,-2,-1,0,0,0,1,2,1).

$$g(x,y)=\sum_{i=-n}^{i=n}\sum_{j=-n}^{j=n} h(i,j).I(x+i,y+j)$$



Fusion : G(x,y)=(Gx(x,y)+Gy(x,y))/2
Seuillage : si G(x,y)>seuil alors ImageN&B(x,y)=0 sinon ImageN&B (x,y)=1.



Exercice : tâches dépendantes

Temps d'exécution : définition image 640x420, fps =30

	T1	T2	T3	T4	T5
Temps	2ms	4ms	4ms	1ms	1ms
Période	30ms	30ms	30ms	30ms	30ms

Peut-on ordonnancer ces tâches sur un core de processeur ?

Que devient le temps d'exécution si on double la définition de l'image et que le fps =50 ?

Dans ce dernier cas, combien de core de processeur sont nécessaire pour ordonnancer (schedule) ces tâches ?